

LESSON 22

Autoencoders

Fine Tuning and Transfer Knowledge

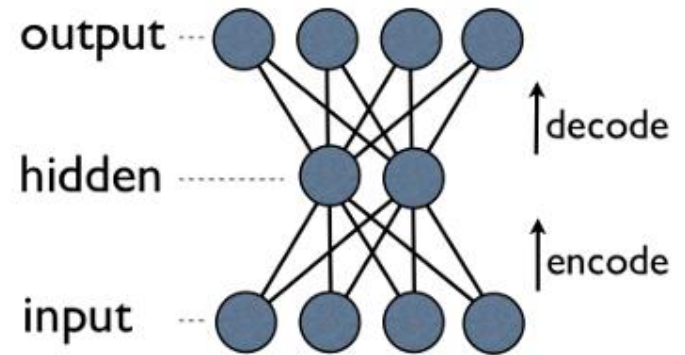
Automatic Feature Extraction

Training of Deep Learning Models



Outline

- Autoencoders
- Training and design of deep learning models
 - Fine Tuning
 - Transfer Knowledge
 - Data augmentation
- Catastrophic forgetting
- Automatic Feature Extraction
- Problems of the training of deep learning models.
- A solution: Greedy Layer-Wise Training
- Generative Adversarial Networks
 - Generation of realistic synthetic images
 - Model autophagy disorder for generative models
- LLM and Design

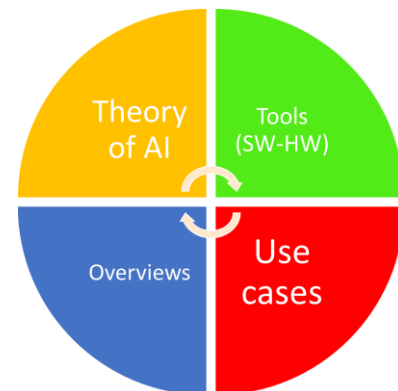




THEORY

Unsupervised deep learning models: Autoencoders

Working in training with no labels



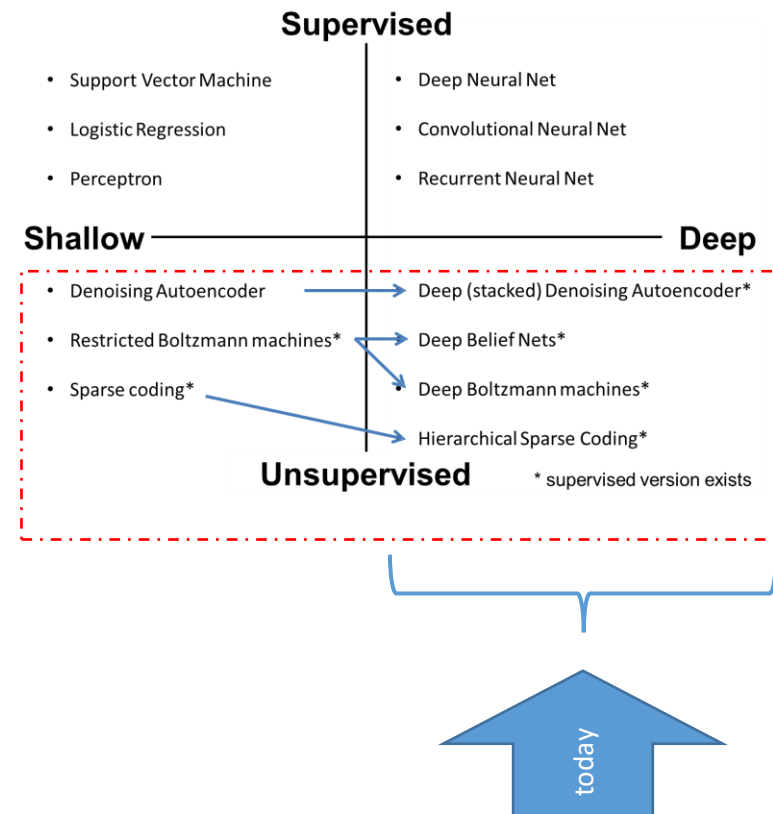
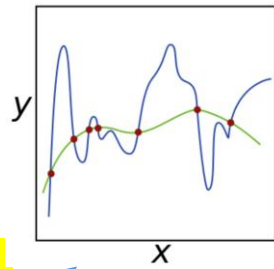
Unsupervised learning

- Can use **unlabeled data** (unlimited)
- Can be **refined with standard supervised techniques** (e.g. backpropagation like a normal feedforward NN)
- Useful when the **amount of labels is small** → partially supervised dataset (semi-supervised)

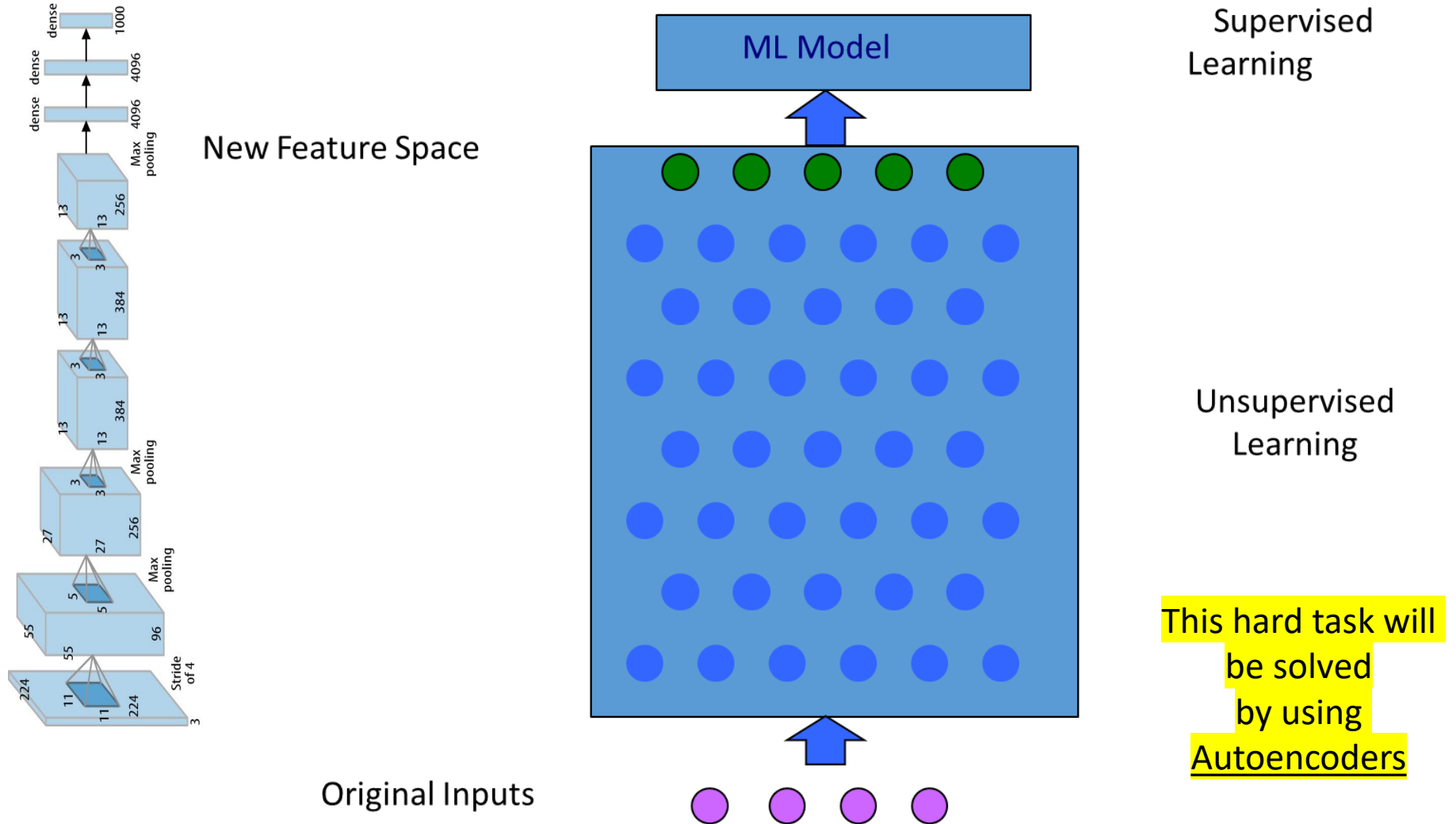


Unsupervised learning for classical and deep learning models

- Main idea: since we have no labels, **methods try to model the distribution of input data** (e.g., the clusters)
 - Reconstruction error
 - Log-likelihood of data
- Models
 - Basic: PCA, KMeans
 - Independent Comp. Analysis
 - Denoising autoencoders
 - Sparse autoencoders
 - Restricted Boltzmann machines
 - Sparse coding
 - ...

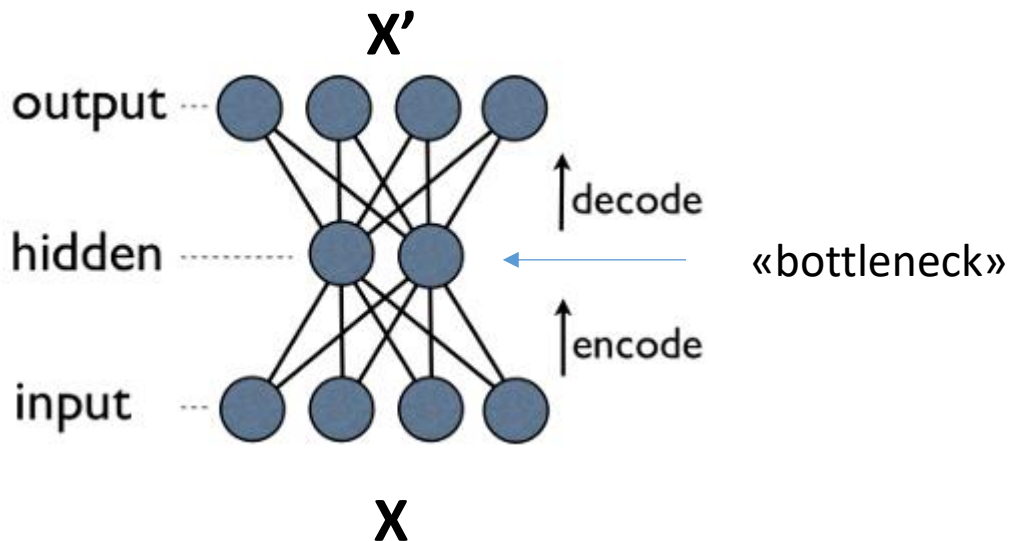


Solution: Autoencoders and Greedy Layer Wise Training



Autoencoders (AEs)

- Try to create generic features from data (encoding)
 - Learn identity function (output = input) by learning important subfeatures
 - Compression, etc.
- We can just use the new features in the new training set or concatenate both.



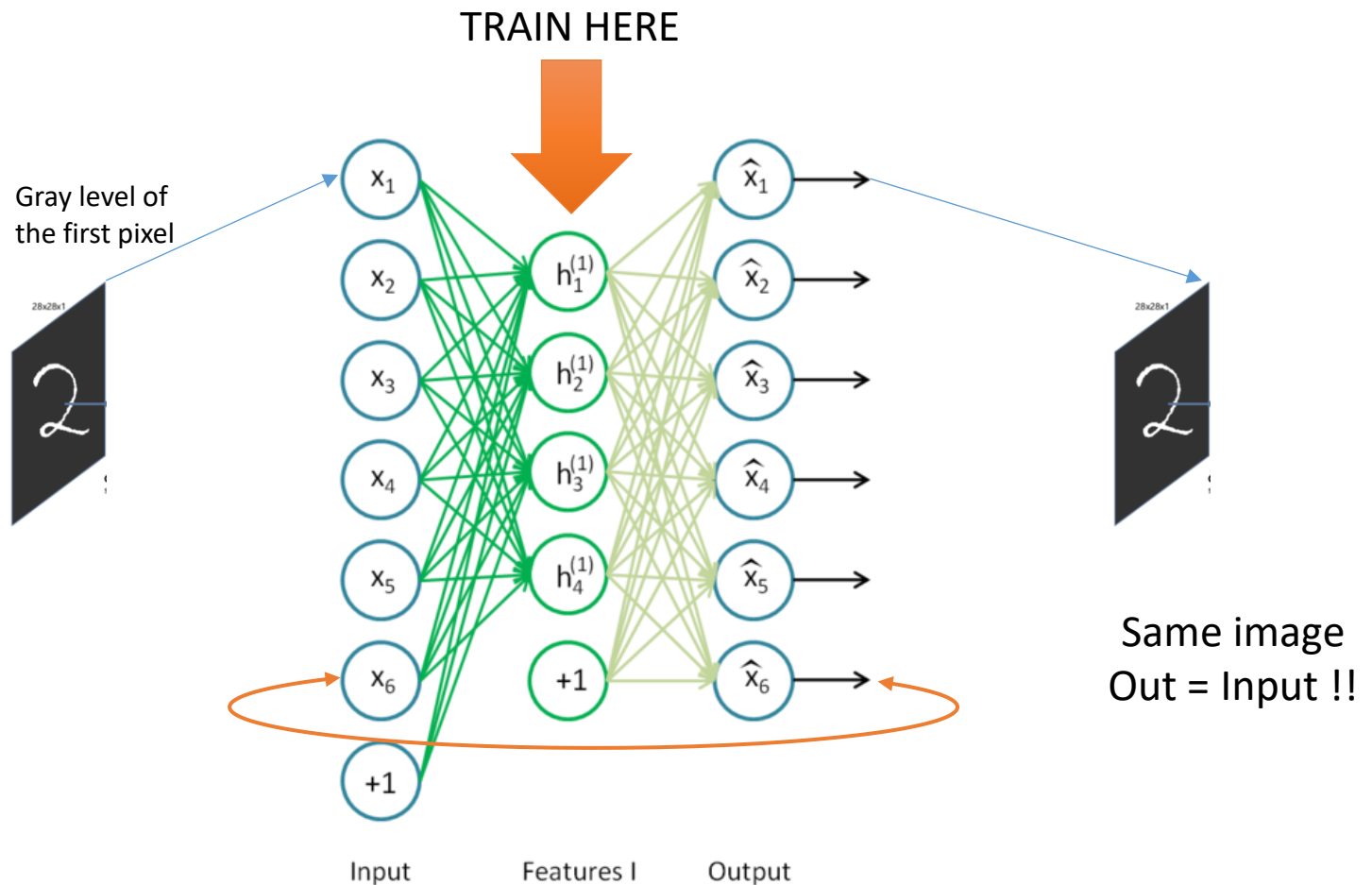
The training aims to

$$X=X'$$

Using backpropagation like a normal feedforward NN

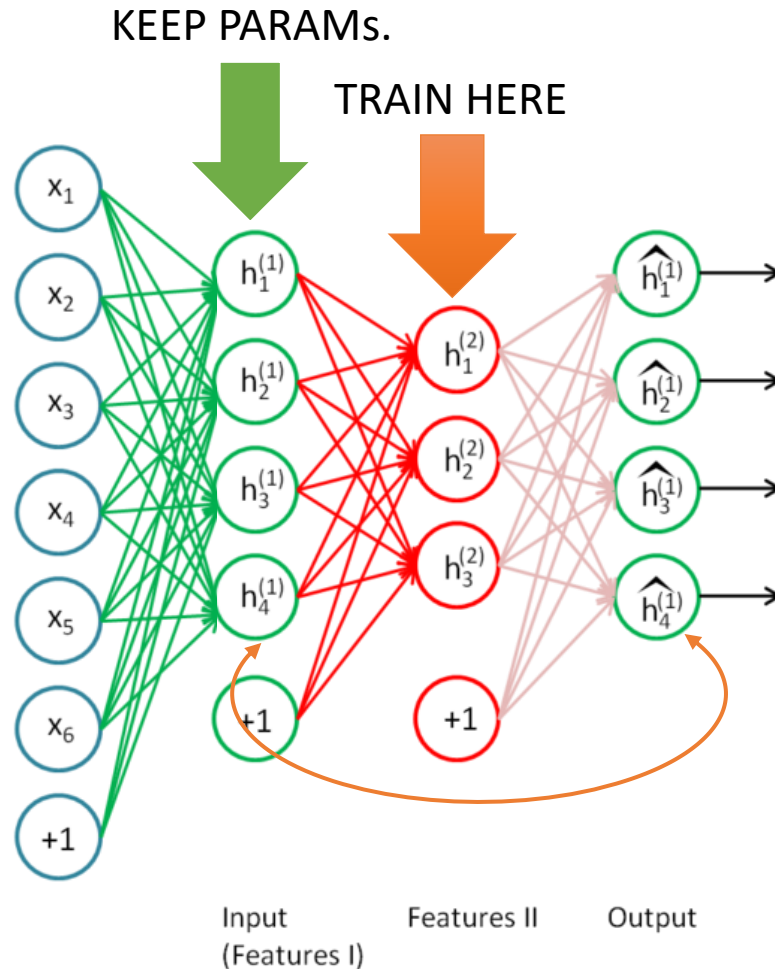
Stacked Auto-Encoders

- Stack many (sparse) auto-encoders in succession and train them using greedy layer-wise training
- Drop the decode output layer each time



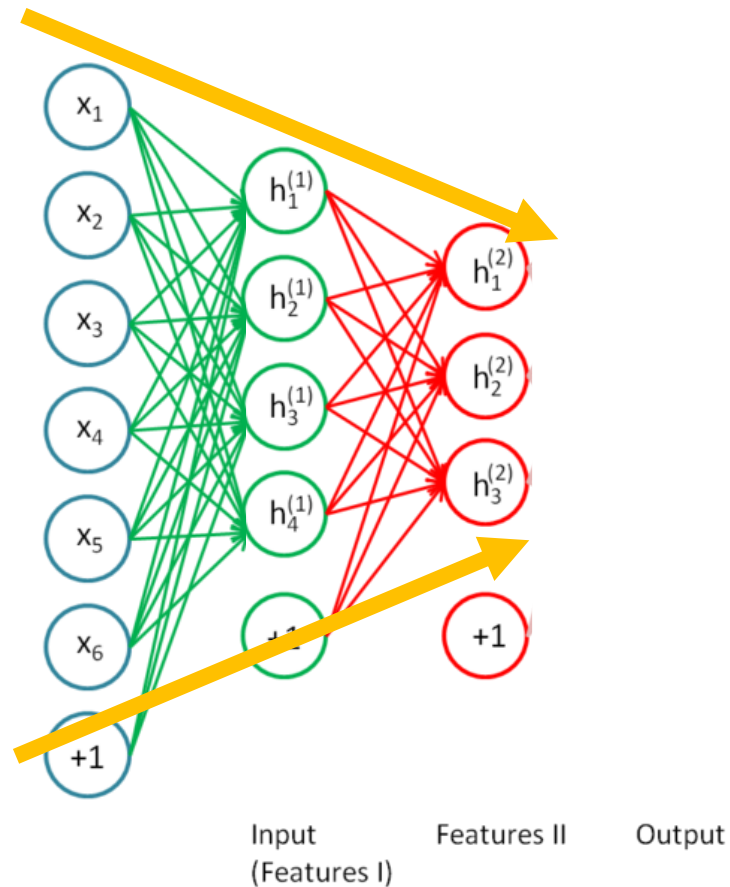
Stacked Auto-Encoders

- Stack many (sparse) auto-encoders in succession and train them using greedy layer-wise training
- Drop the decode output layer each time



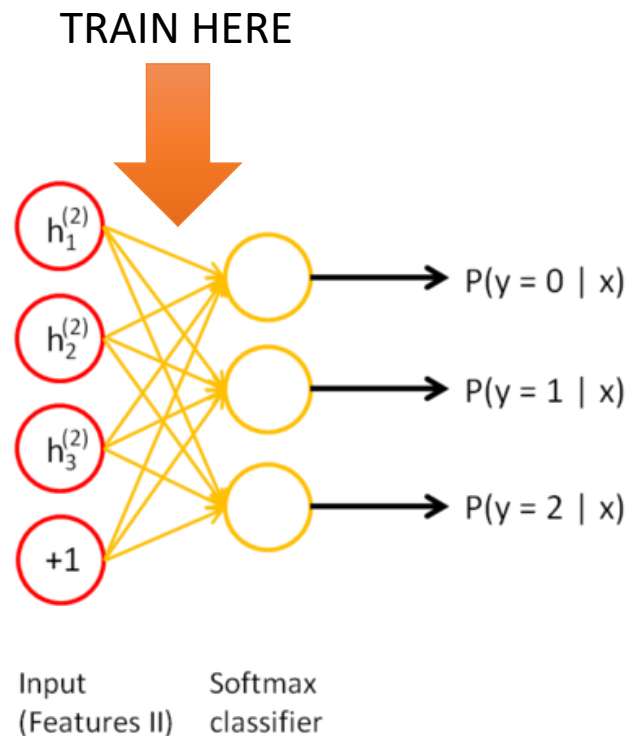
Stacked Auto-Encoders

- The golden rule is to reduce the number of neurons going forward to the outputs



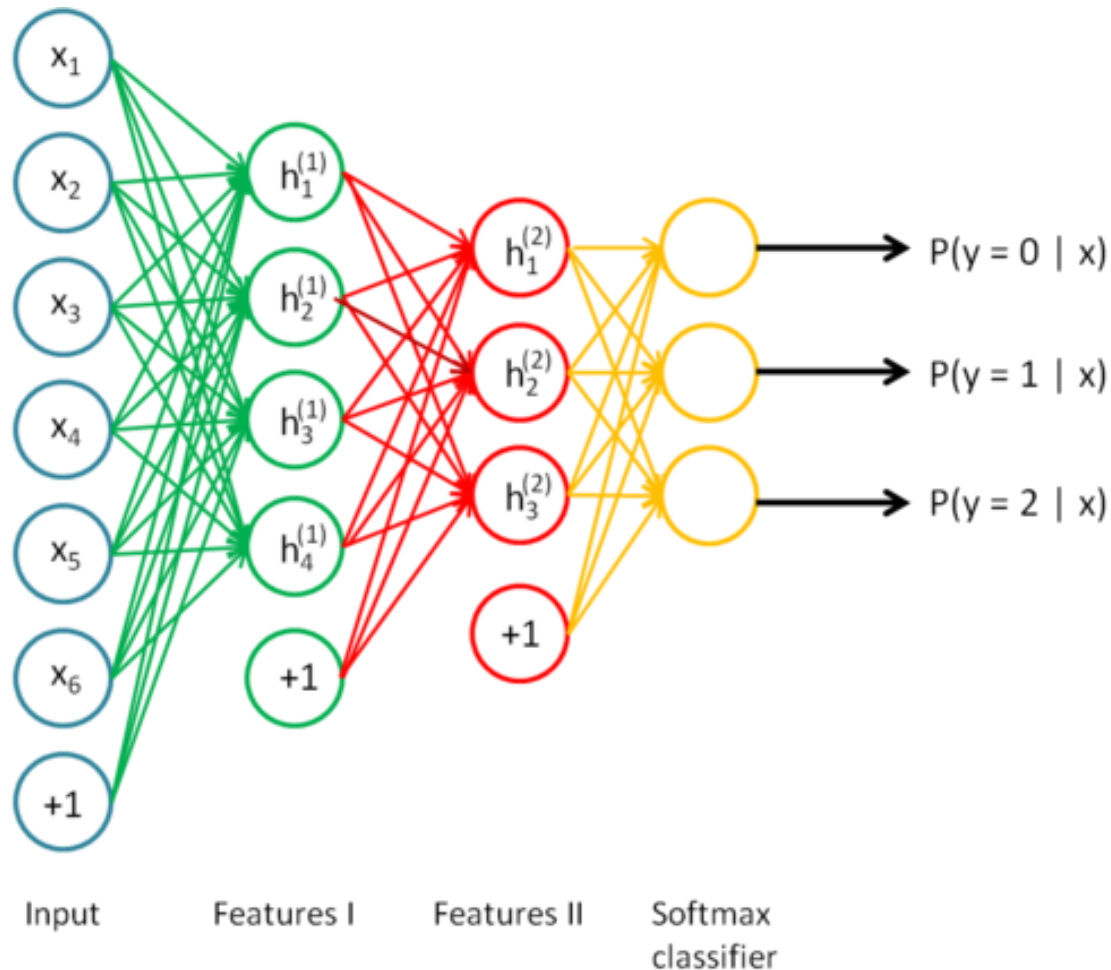
Stacked Auto-Encoders (2)

- Do **supervised training** on the last layer.
We can do it since we have
 - The inputs X and the weights of the AEs $\rightarrow$ we can process the inputs of the final layer ($h_1, h_2, \dots$)
 - We have the labels in the Y output vector



Stacked Auto-Encoders (3)

- Then do **supervised training on the entire network** to fine-tune all weights

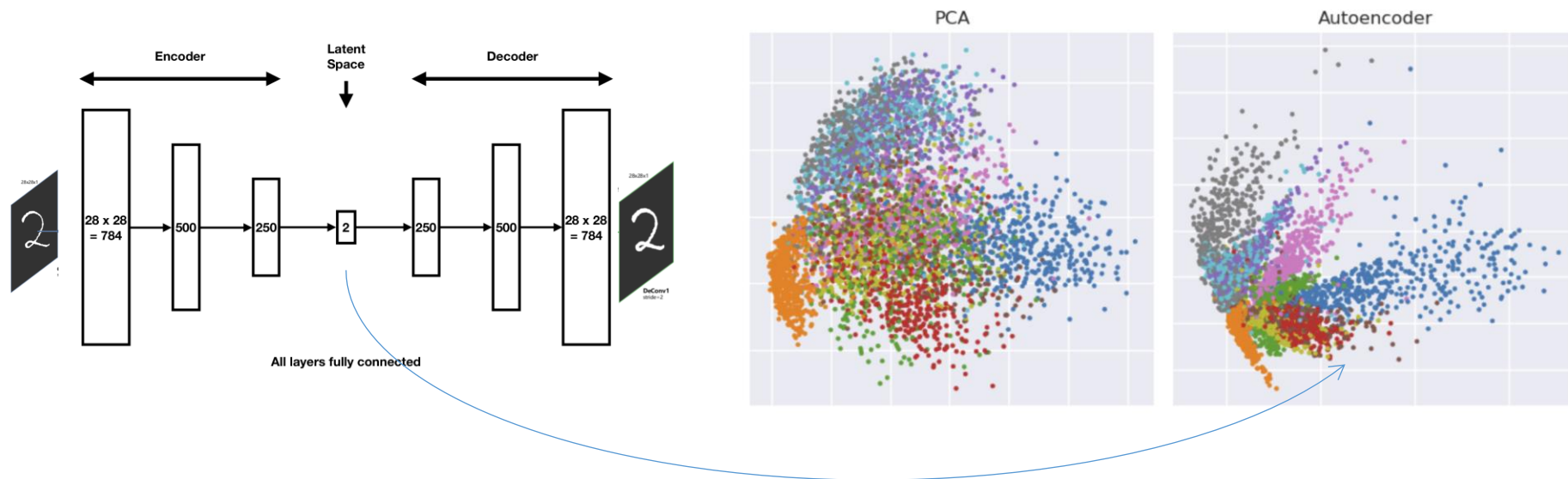


State of the art

- Greedy layer-wise training with autoencoders was very important historically to initialize deep networks before modern training recipes became robust.
- Today it is less common for standard CNNs and transformers, because we usually rely on
 - better initialization, normalization,
 - residual connections,
 - large datasets,
 - pretrained models,
 - transfer learning.
- In modern CNN training, the early layers are usually not trained from scratch with autoencoders. They are either learned end-to-end using robust architectures, or reused from pretrained models and kept frozen or lightly fine-tuned.

Applications of AEs: Dimensionality reduction

Example from the MNIST dataset ($28 \times 28 \times 28 \times 28$ black and white images of single digits) from the original 784 dimensions to 2

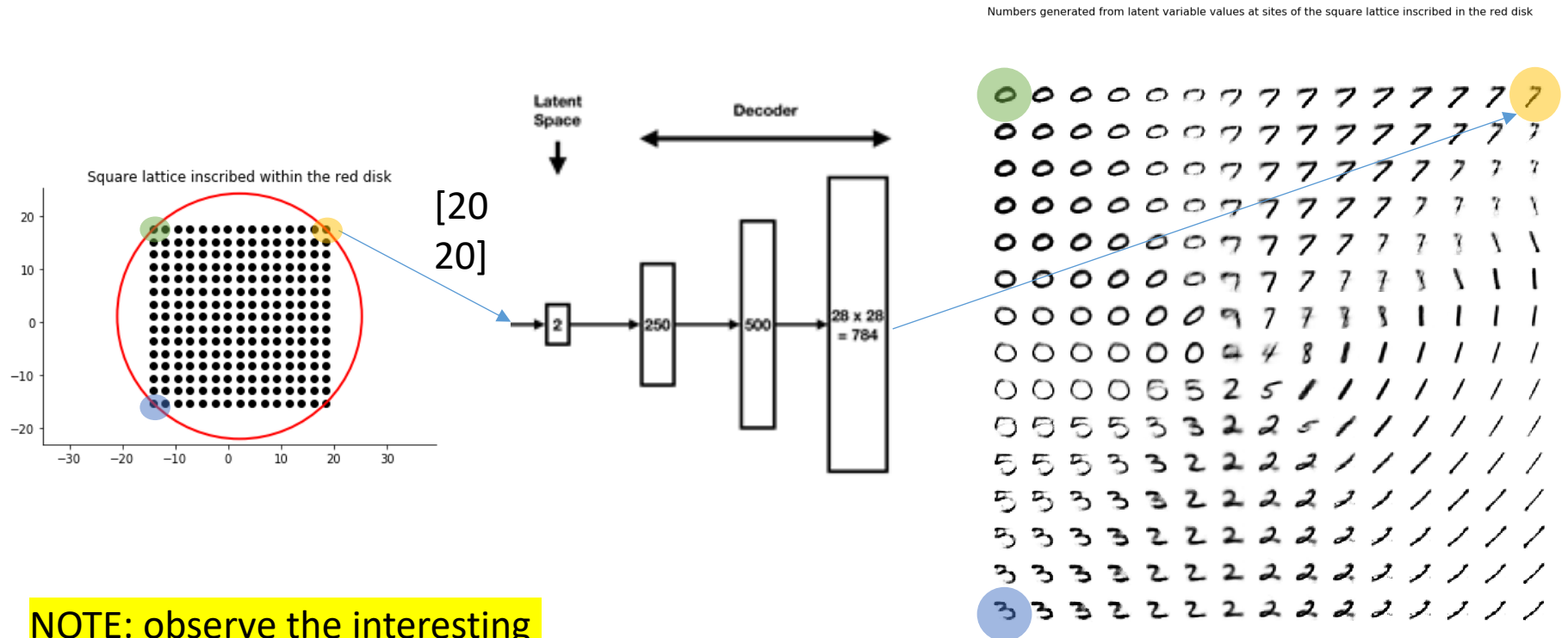


Note: PCA is an unsupervised method as AEs

Understanding AEs:

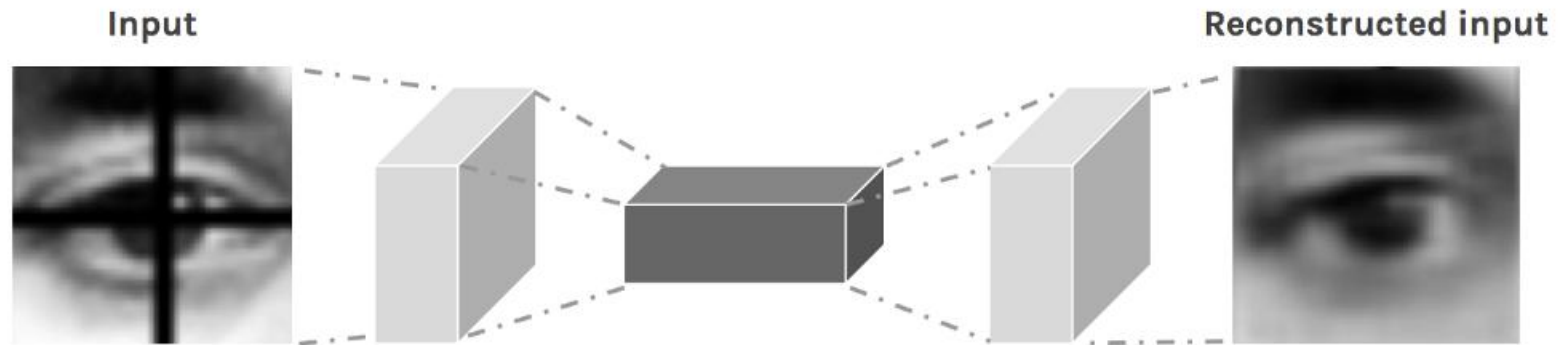
After training, we can use just the decoder

Let's try with different 2D input vectors, for example the input [20,20] will produce the output image 

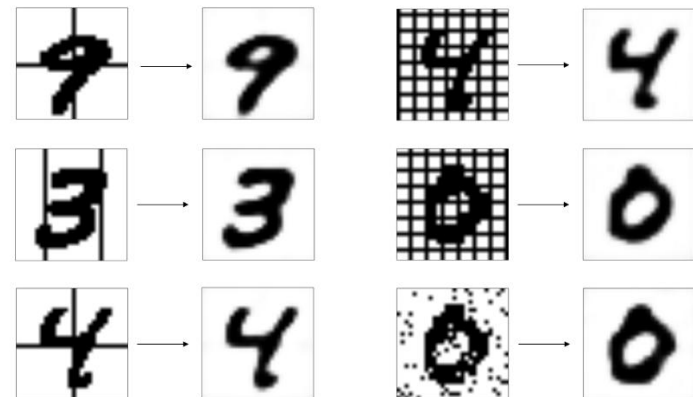
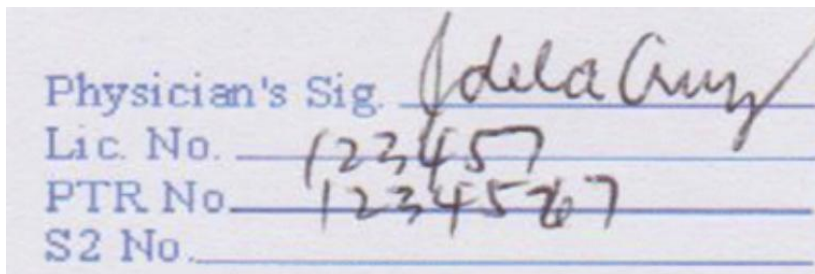


NOTE: observe the interesting continuity of the latent space!

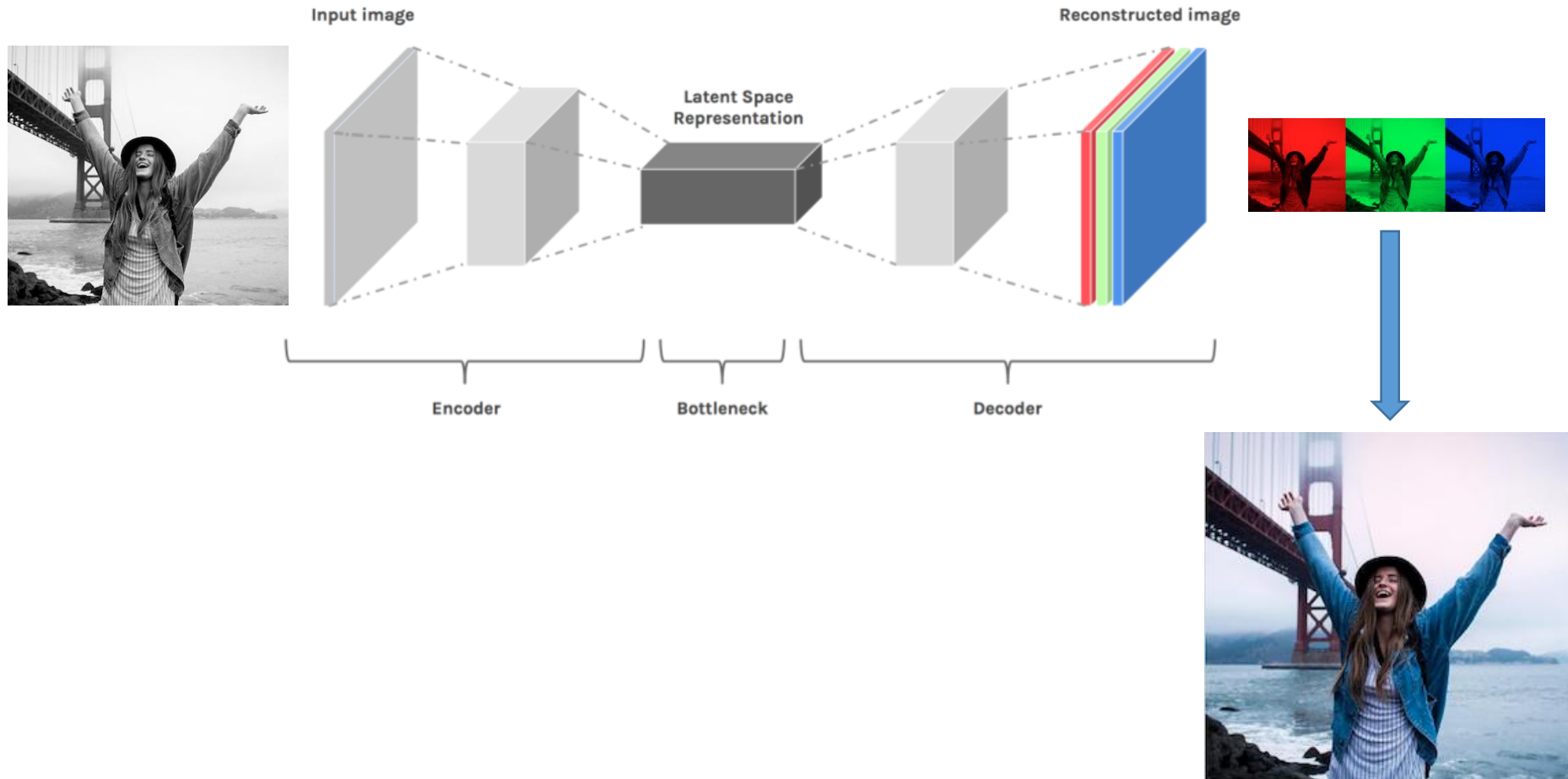
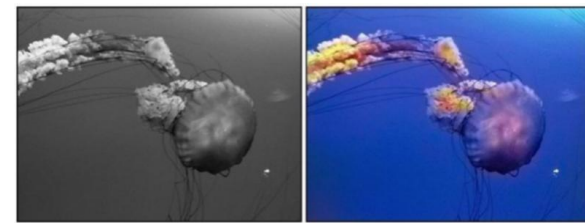
Applications of AEs: Noise reduction



Missing pixels, noise, bars



Applications of AEs: Auto Colorization

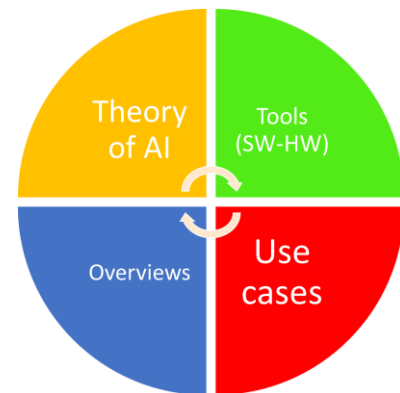




THEORY

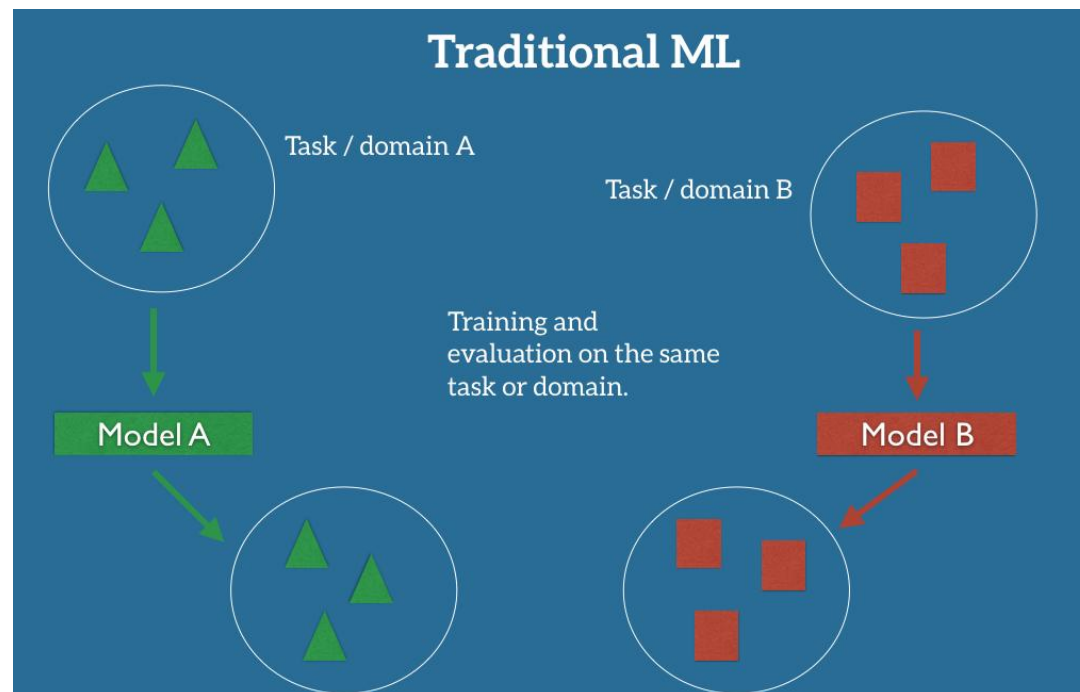
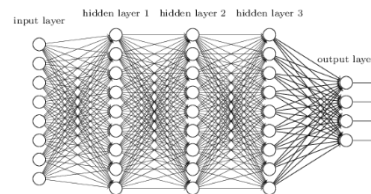
Transfer learning and fine-tuning in deep models

Exploit previous training and knowledge

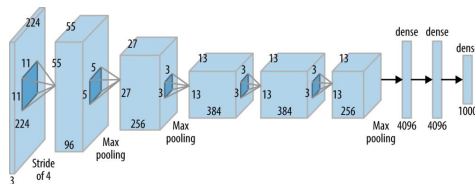


Transfer Learning to avoid overfitting and improving learning

Traditional ML methods need
to start over every time the learning
process when you change the application
domain/datasets



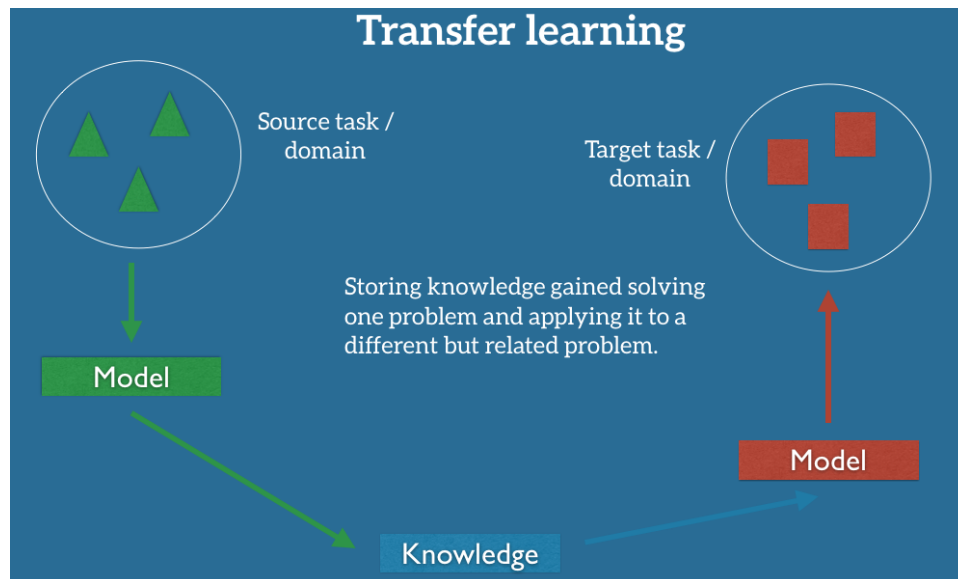
Transfer Learning to avoid overfitting and improving learning



Domain 1



Step1 no background
→ Better learning



Domain 2



Step2 more
complex scenes
Can be faced
during learning

Pretrained models are available online

- Large CNNs take weeks to train across multiple GPUs on image dataset like ImageNet
- Many researcher release their final CCN for the benefit of others who can use the networks for fine-tuning.
- Main sources
 - Hugging Face Hub
 - Keras Applications / KerasHub
 - PyTorch Torch Vision models
 - TensorFlow Hub
 - ONNX Model Zoo

FINE TUNING

starting from a public CNN

Classification PB1:

Face, body, dog,
horse, ...

Classification PB2:

FACE YES/NO



Convolution

Max Pooling

Convolution

Max Pooling

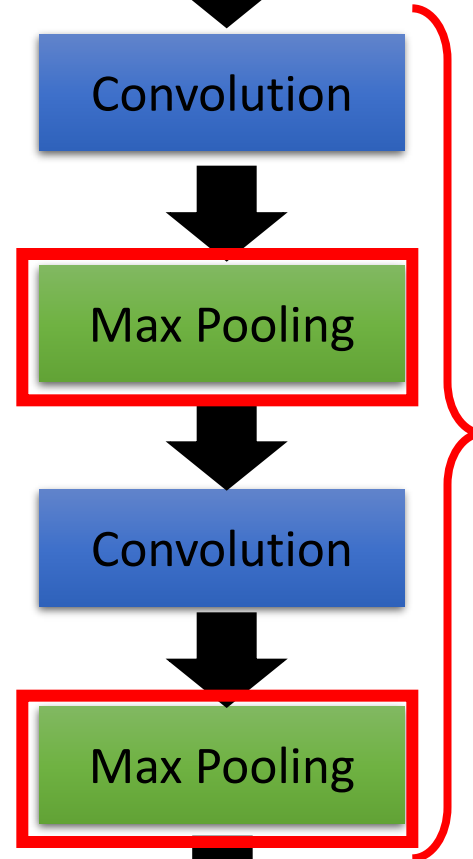
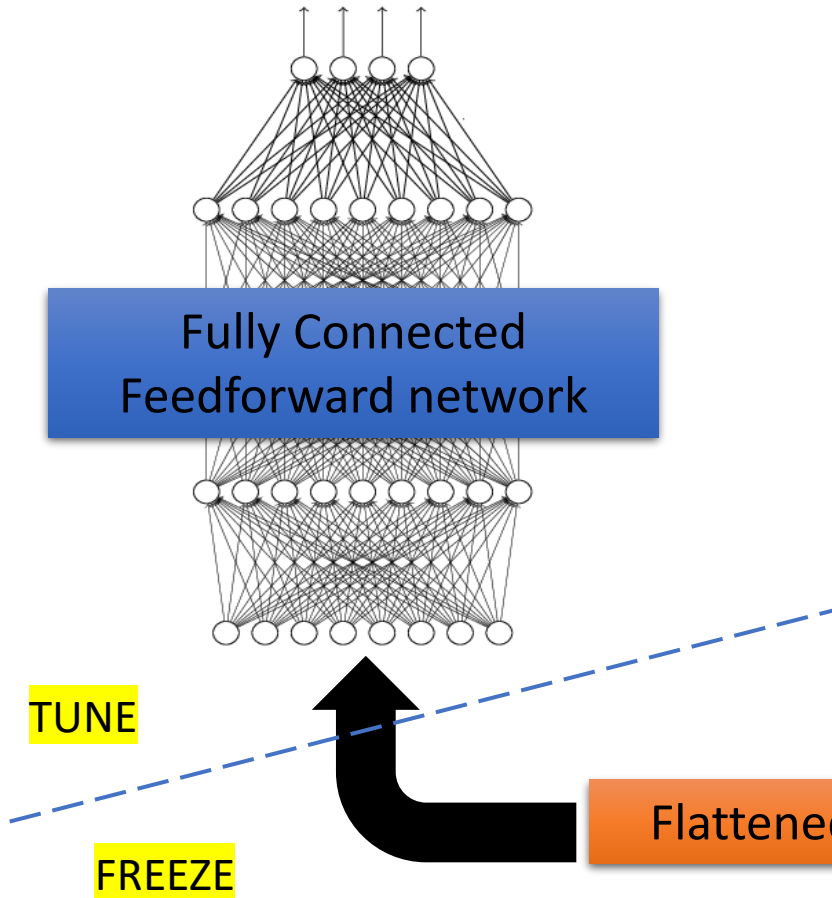
Can
repeat
many
times

Fully Connected
Feedforward network

Flattened

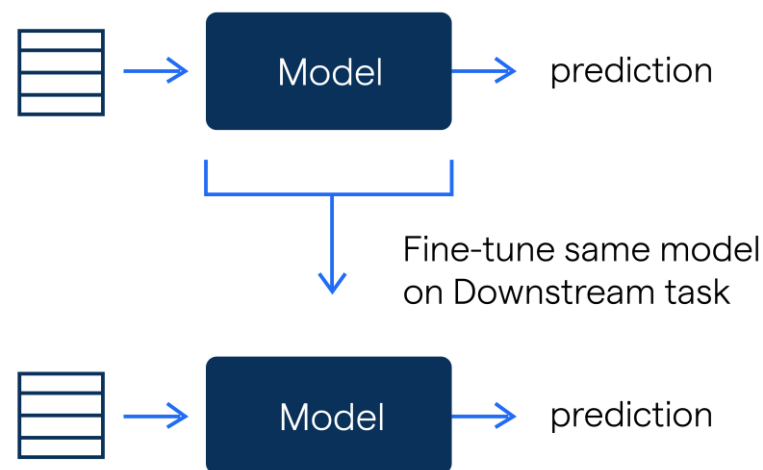
TUNE

FREEZE

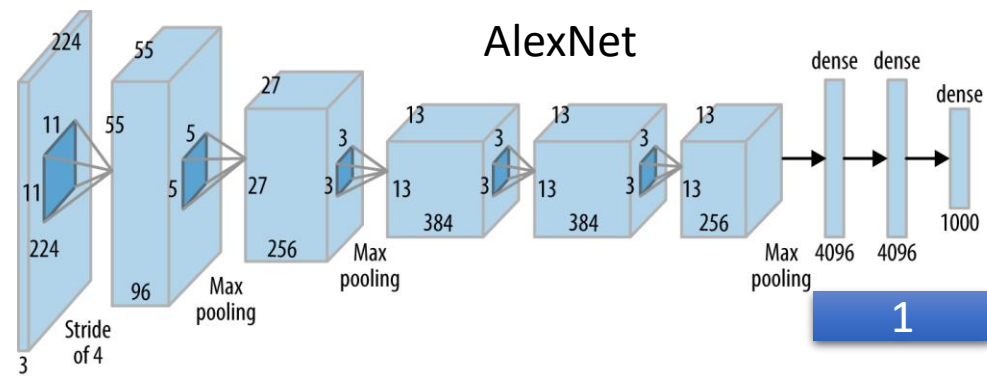


Finetuning is very effective!

- Fine-tuning is now one of the dominant practical workflows in deep learning.
- The same idea applies not only to CNNs, but also to
 - vision transformers,
 - multimodal models,
 - speech models,
 - and LLMs.

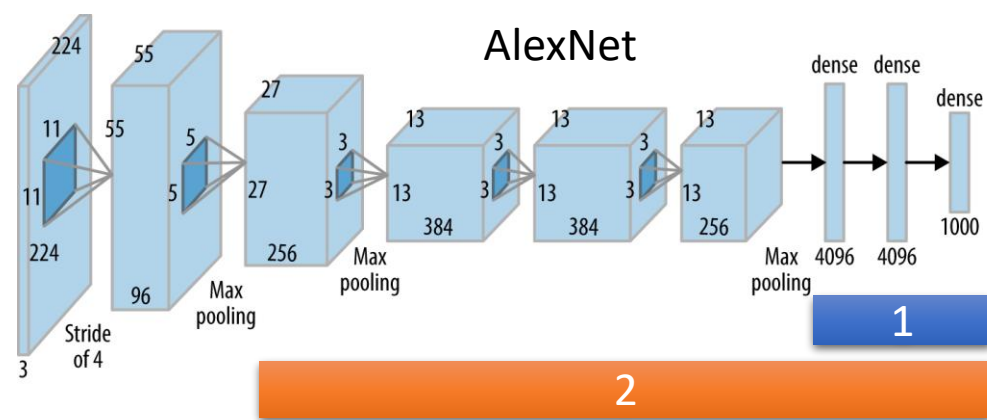


Fine-tuning the CCN (2)



- Options
 - 1) replace and **retrain the classifier on top** of the ConvNet on the new dataset,

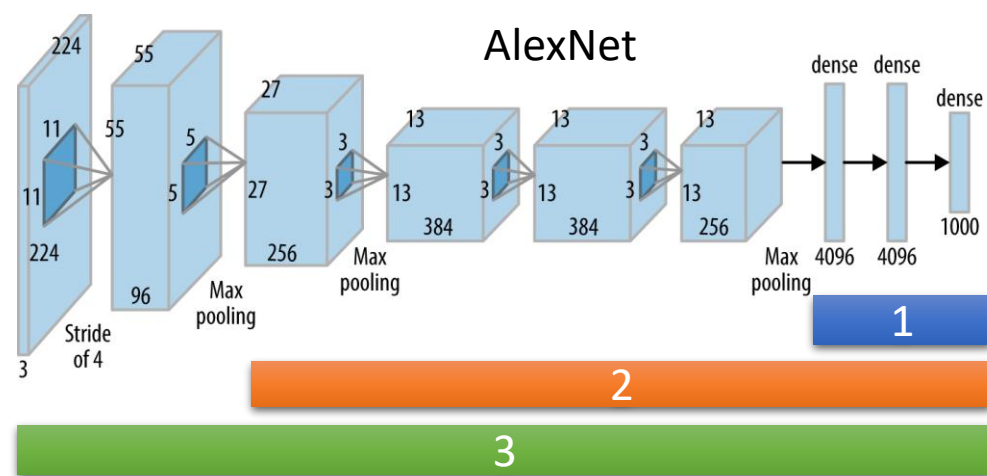
Fine-tuning the CCN (2)



- Options

- 1) replace and retrain the classifier on top of the ConvNet on the new dataset,
- 2) Keep some of the earlier layers fixed (due to overfitting concerns) and only fine-tune some higher-level portion of the network.
 - This is motivated by the observation that the earlier features of a ConvNet contain more generic features (e.g. edge detectors or color blob detectors) that should be useful to many tasks, but later layers of the ConvNet becomes progressively more specific to the details of the classes contained in the original dataset.

Fine-tuning the CCN (2)



Options

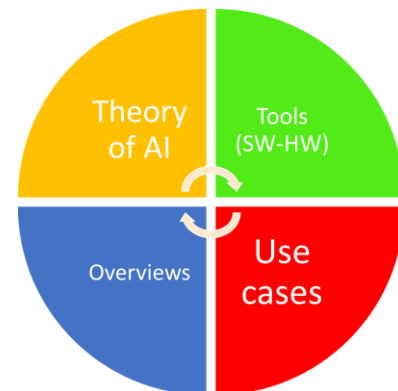
- 1) replace and **retrain the classifier on top** of the ConvNet on the new dataset,
- 2) **Keep some of the earlier layers fixed** (due to overfitting concerns) and only fine-tune some higher-level portion of the network.
 - This is motivated by the observation that the earlier features of a ConvNet contain more generic features (e.g. edge detectors or color blob detectors) that should be useful to many tasks, but later layers of the ConvNet becomes progressively more specific to the details of the classes contained in the original dataset.
- 3) **fine-tune all the weights** of the pretrained network by continuing the backpropagation.



THEORY

Catastrophic forgetting

forgetting previously learned information
when learning new information



Beware!

Catastrophic forgetting/inference



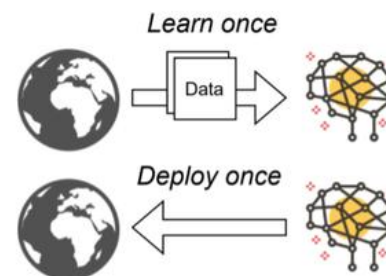
Refers to the phenomenon in machine learning where a neural network forgets previously learned information when it learns new information, resulting in a significant loss of knowledge

Beware!

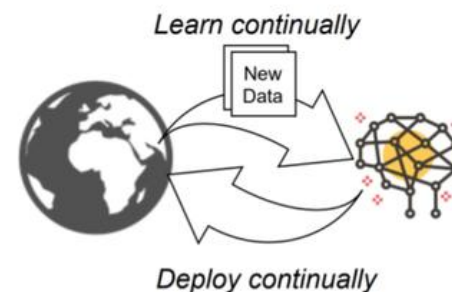
Catastrophic Forgetting



Static ML



Adaptive ML



- A phenomenon observed in ML specifically in **neural networks**
- The ability to remember previously learned information is severely impacted when it learns new information
- This occurs **when a network is sequentially trained on different tasks**, and as it learns new tasks, the network's performance on previously learned tasks **degrades significantly**

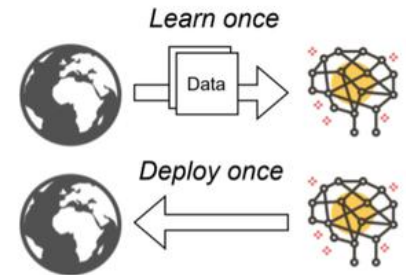
Catastrophic Forgetting



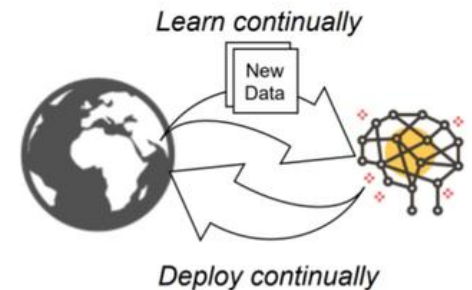
Catastrophic forgetting is now especially relevant in

- continual learning,
- personalization,
- domain adaptation,
- fine-tuning of large pretrained models (also LLMs and LMMs)

Static ML

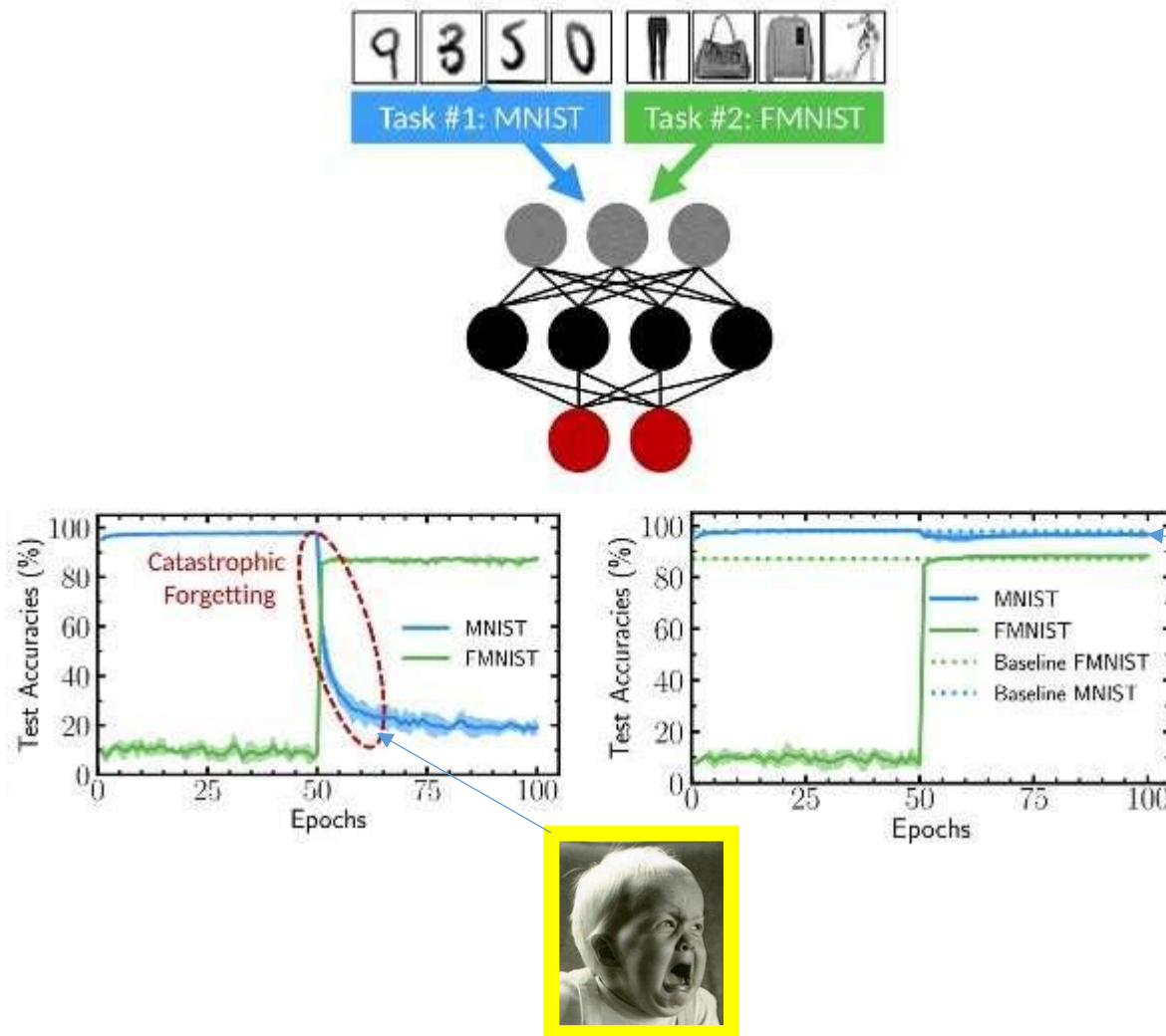


Adaptive ML



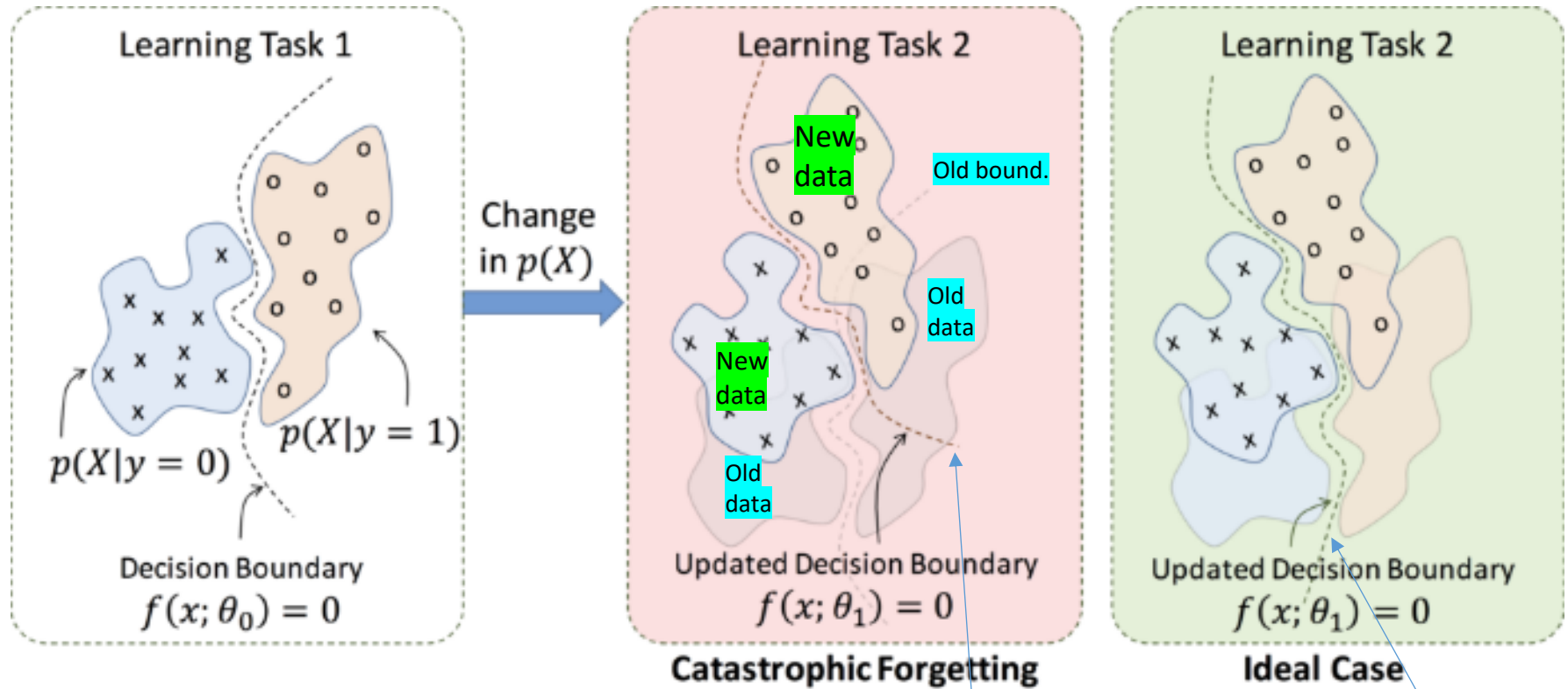
Catastrophic Forgetting

(by validation accuracy)

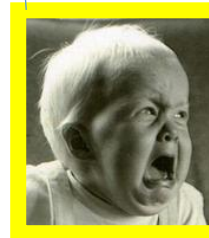


Catastrophic Forgetting

(by Decision Boundaries)



The new boundary
is forgetting how to separate
the old data



Mitigation of Catastrophic Forgetting

It's still an open problem in AI research, there is no complete solution. You can try with

- **Rehearsal**

A straightforward and intuitive strategy, where old data is mixed with the new data during training to help the model remember old tasks. However, this method can be resource-intensive as it requires storing old data.

- **Elastic Weight Consolidation (EWC)**

EWC is a popular technique because it adds a simple regularization term to the loss function, which helps preserve the weights that are important for old tasks

Mitigation of Catastrophic Forgetting

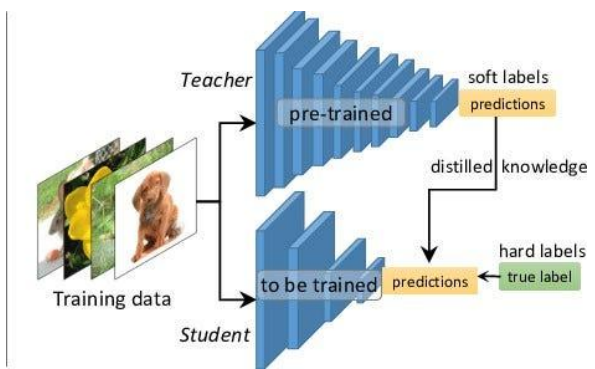
It's still an open problem in AI research, there is no complete solution. You can try with

- **Knowledge Distillation**

The new model (student) is trained to match the output of the old model (teacher), which helps it retain knowledge from old tasks

- **Dynamic Architectures**

Methods like "PathNet" or "Progressive Neural Networks" adjust the network architecture for new tasks. These methods are more complex but can be effective at reducing catastrophic forgetting

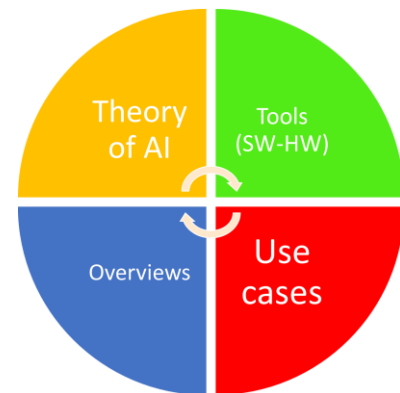




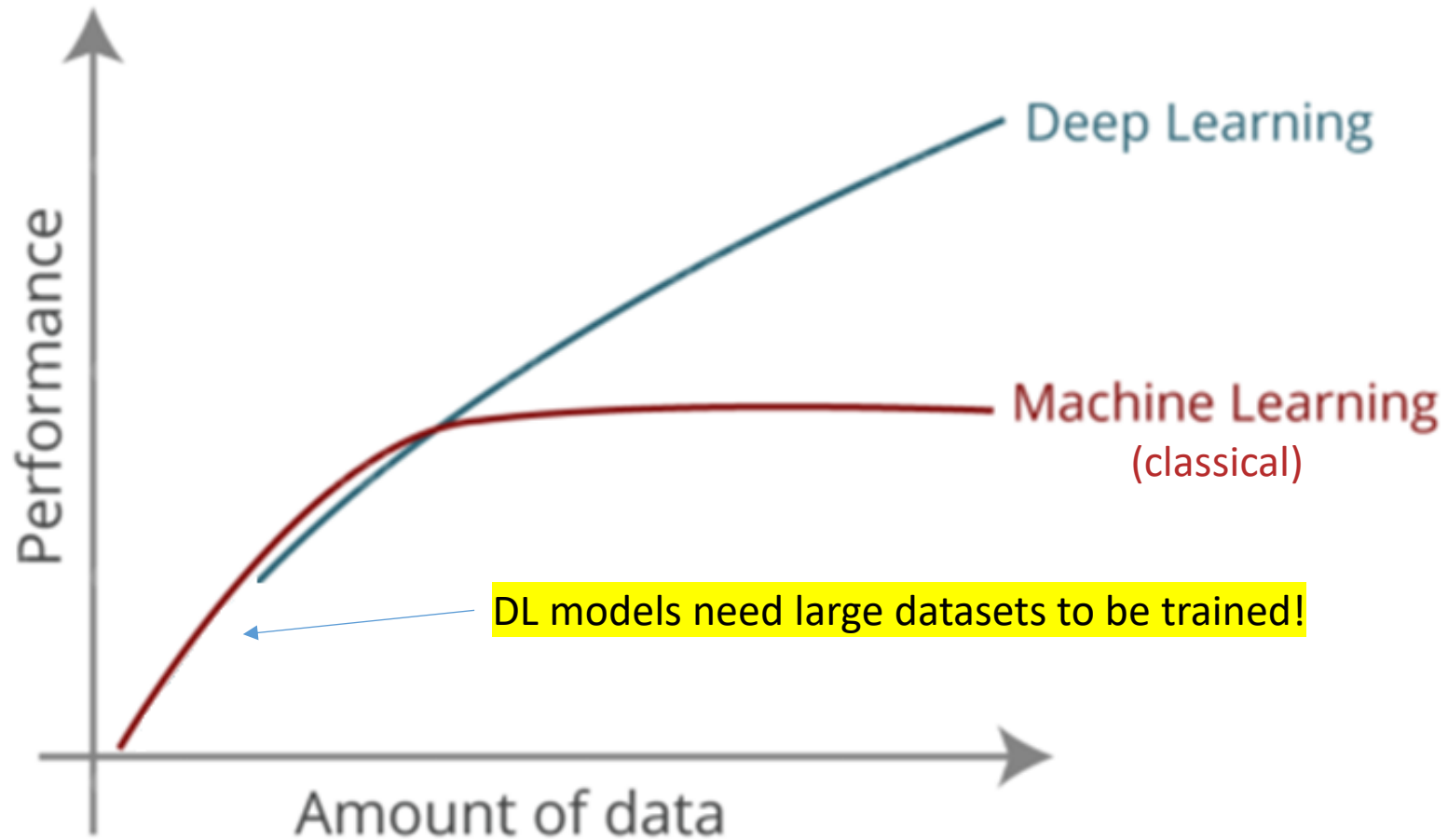
THEORY

Data augmentation

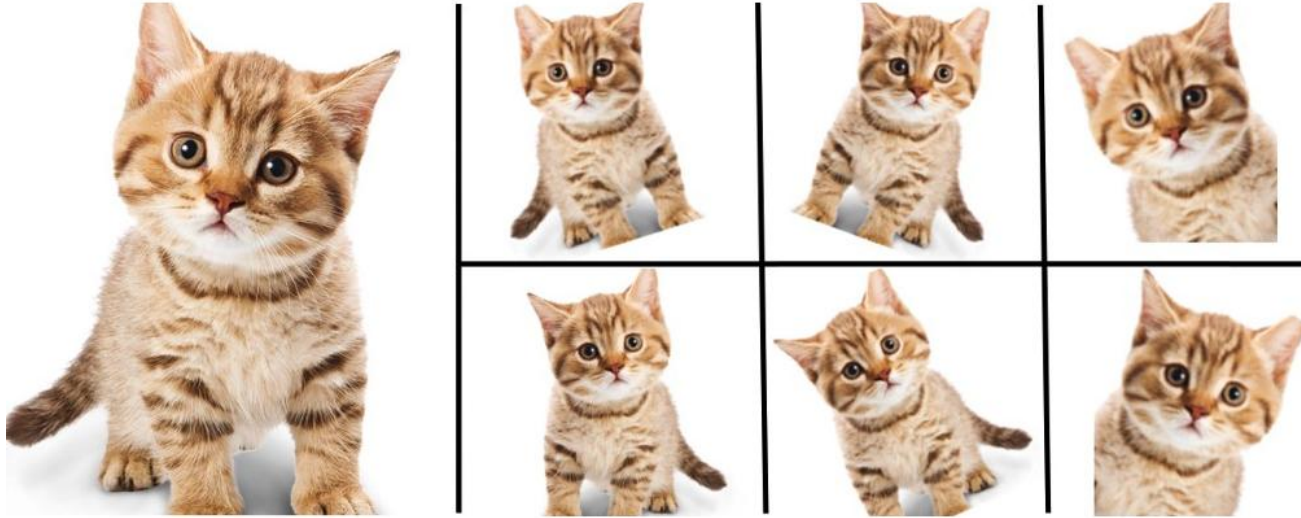
Improving/enlarging the training dataset



Classical ML vs DeepLearning



Data augmentation



Enlarge your Dataset

- **YOUR ALLAY TO OVERCOME OVERFITTING PROBLEMS!**
Please keep in mind that you can have >100M free parameters to fix/tune!
- Data augmentation will not solve the application by itself: **you still need good real data**

Data augmentation #1: rotations, translations, scales



Same tennisball input image.

We let the CNN learn to
detect the object in different

→ positions

→ scales

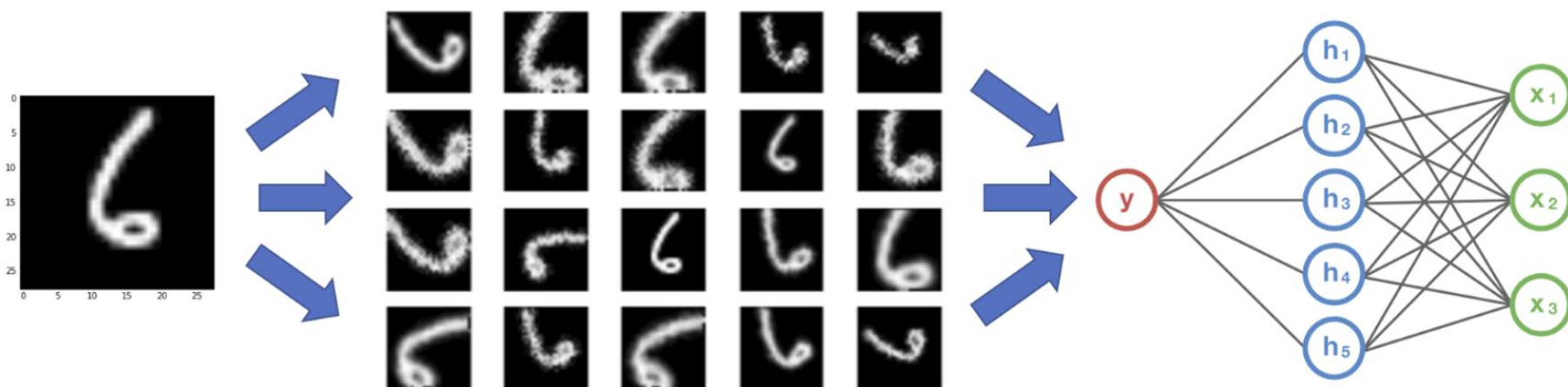
→ rotations

Example: the Matlab `imageDataAugmenter` is randomly flipping the training images along the vertical axis, and randomly translating them

```
pixelRange = [-30 30];  
imageAugmenter = imageDataAugmenter( ...  
    'RandXReflection',true, ...  
    'RandXTranslation',pixelRange, ...  
    'RandYTranslation',pixelRange);  
augImdsTrain = augmentedImageDatastore(inputSize(1:2),imdsTrain, ...  
    'DataAugmentation',imageAugmenter);
```

Advanced toolboxes have specific functions to create this kind of data augmentation.

Data augmentation #2: with noise/noises



Noise of different types (gaussian, additive, multiplicative),
application conditions, sensor noise (pixel noise),
resolutions, environments (indoor, outdoor, backgrounds), ...

Data augmentation #3:

with different (simulated) conditions
(even for big datasets)

You should consider to reproduce real conditions in training

You do it using
public CNNs!

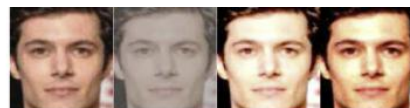
Landmark
perturbation
for face
alignment.



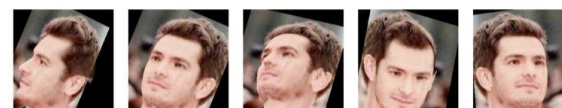
Flipping
clipping
color casting
blurring



(a)



(b)



(c)



(d)



(e)

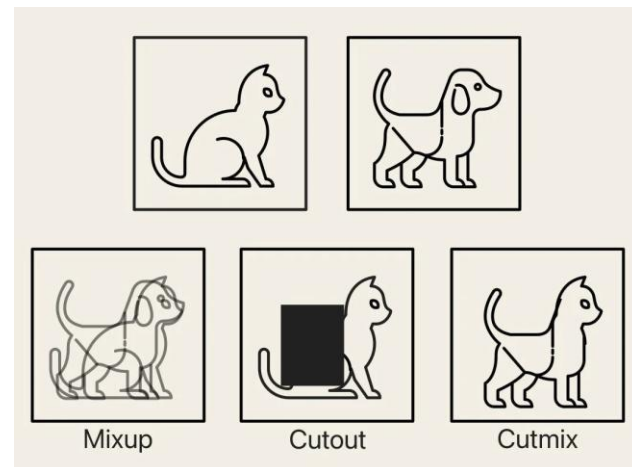


(f)



Data augmentation S0A

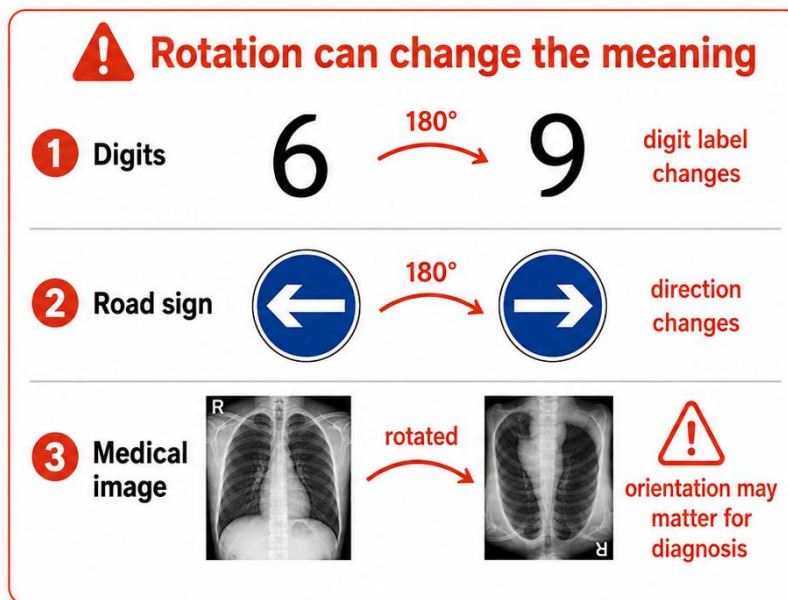
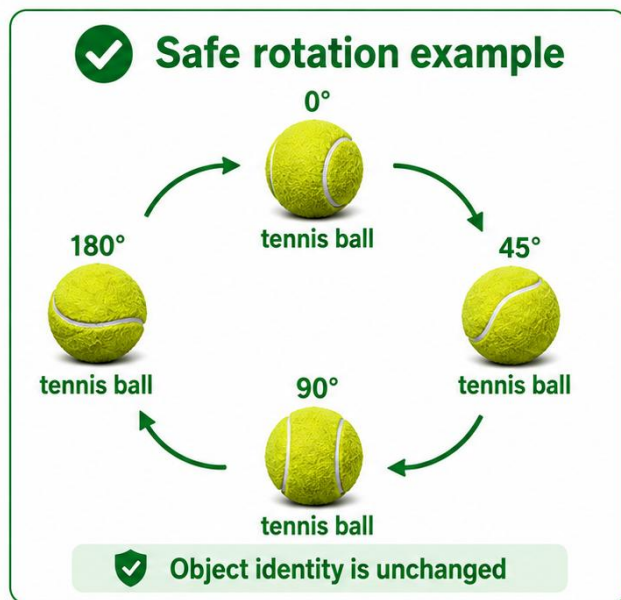
- Modern augmentation also includes
 - CutMix, MixUp, Cutout
 - RandAugment,
 - AutoAugment,
 - Synthetic data generation, domain randomization,
 - Generative augmentation.



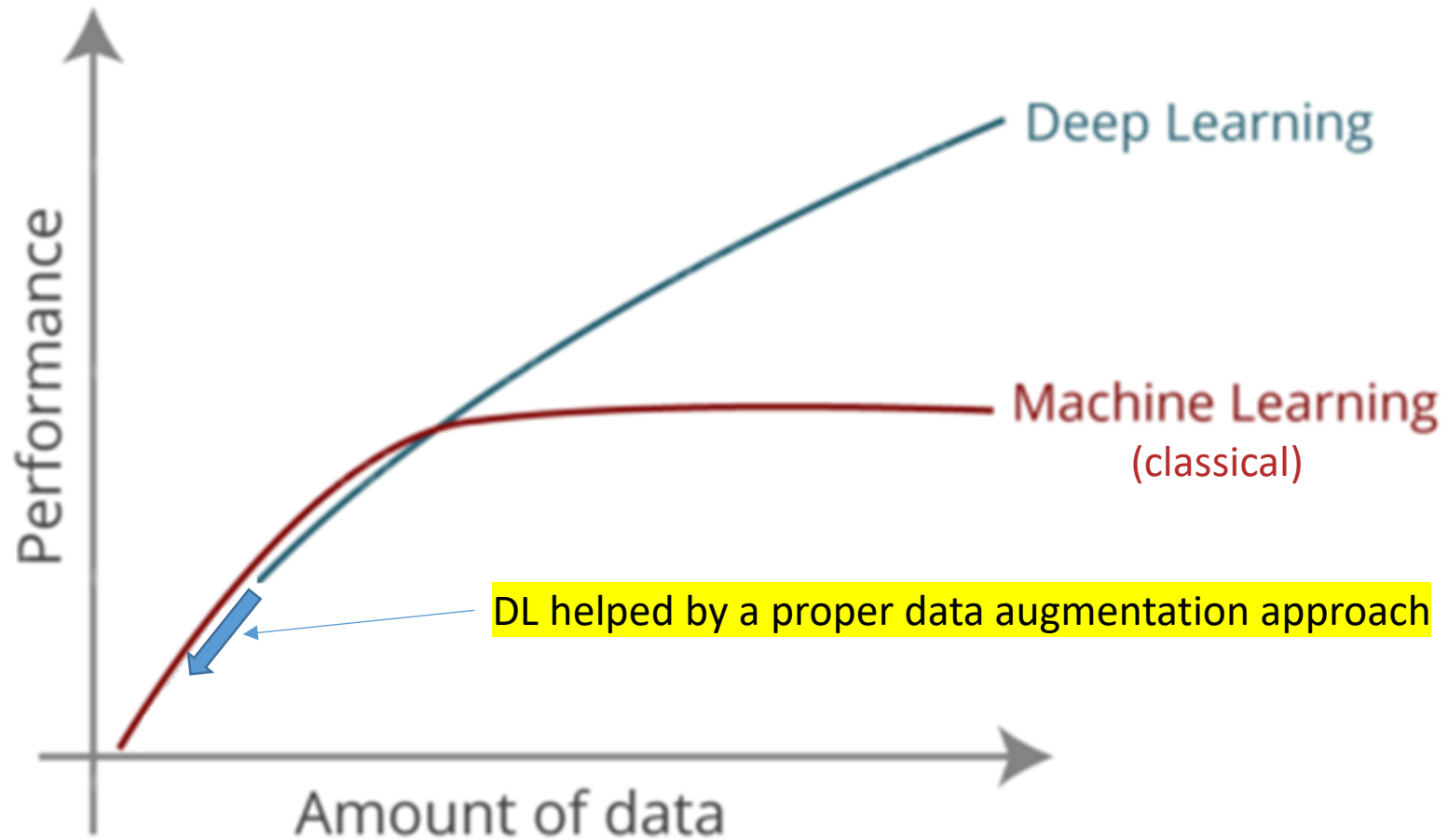
Caveat!

Augmentation must preserve the label!

- A rotation is useful for a tennis ball detector, but may be wrong for digits 6 and 9, road signs, medical images, or orientation-sensitive defects.



DeepLearning + Data Augm.

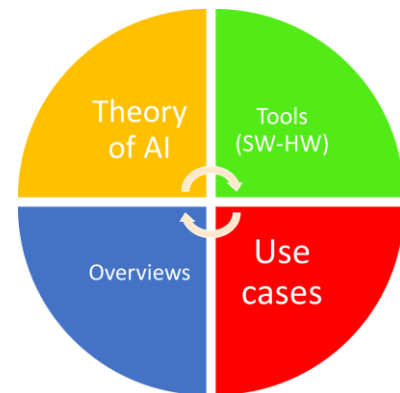




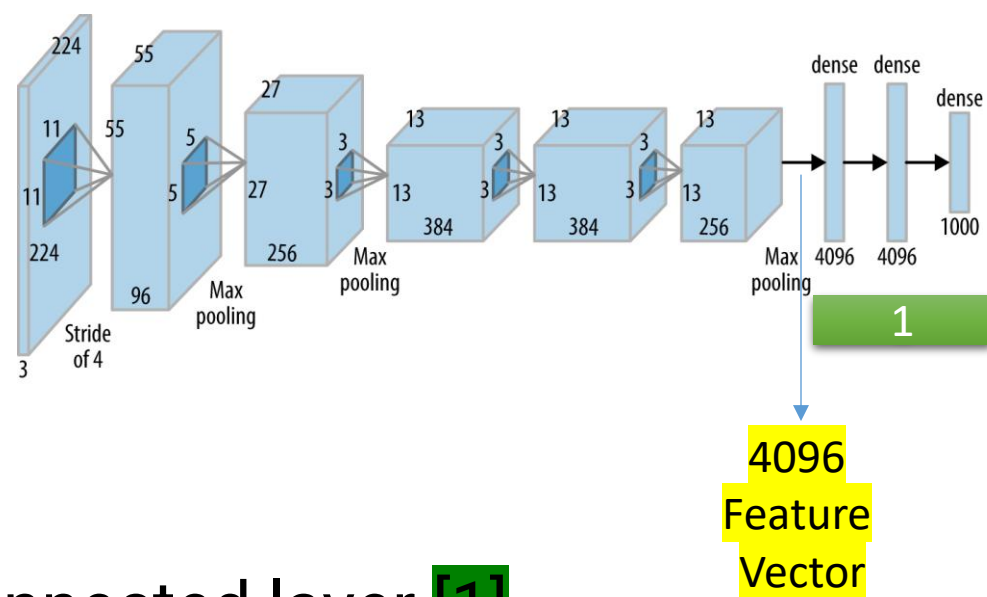
THEORY

Automatic Feature Extraction with deep learning models

Improving the PCA approach using DL

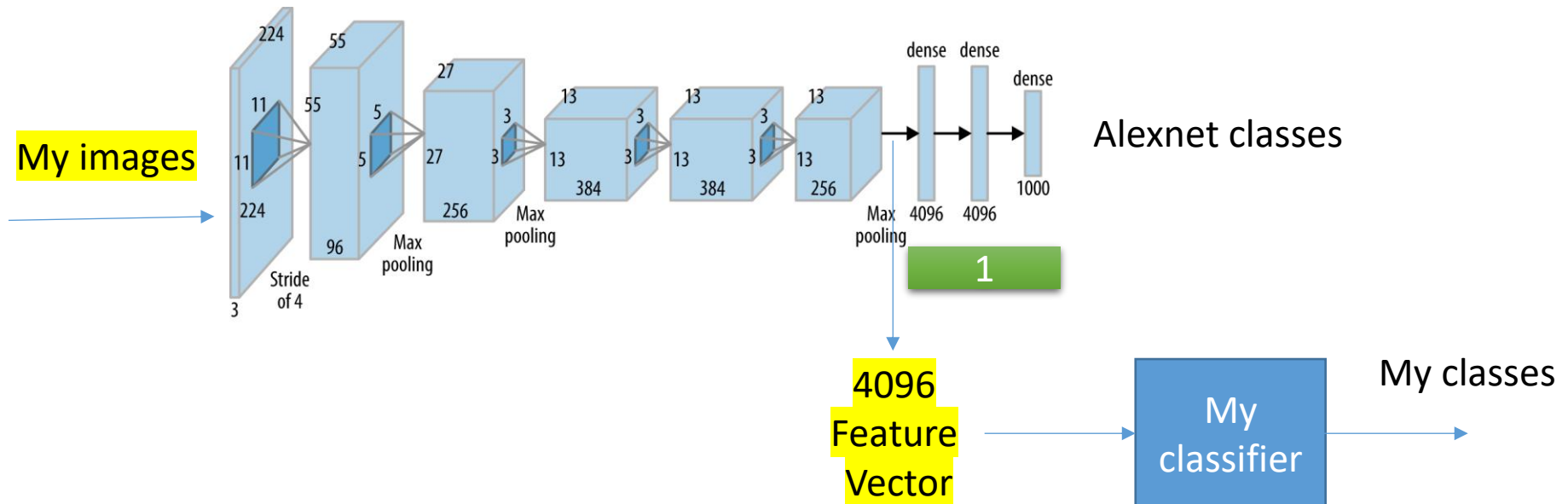


CNNs as feature extractors



1. In a pretrained CNN remove the last fully-connected layer [1]
 - For example, this layer's outputs are the 1000 class scores for a different task like ImageNet
2. Then treat the rest of the CNN as a fixed feature extractor for the new dataset.
 - In an AlexNet, this would compute a 4096-D vector for every image that contains the activations of the hidden layer immediately before the classifier.
 - Note: It is important to note if these features are "ReLUed" (i.e. thresholded at zero) if they were also thresholded during the training of the CNN (as is usually the case).

ConvNets as feature extractors (2)

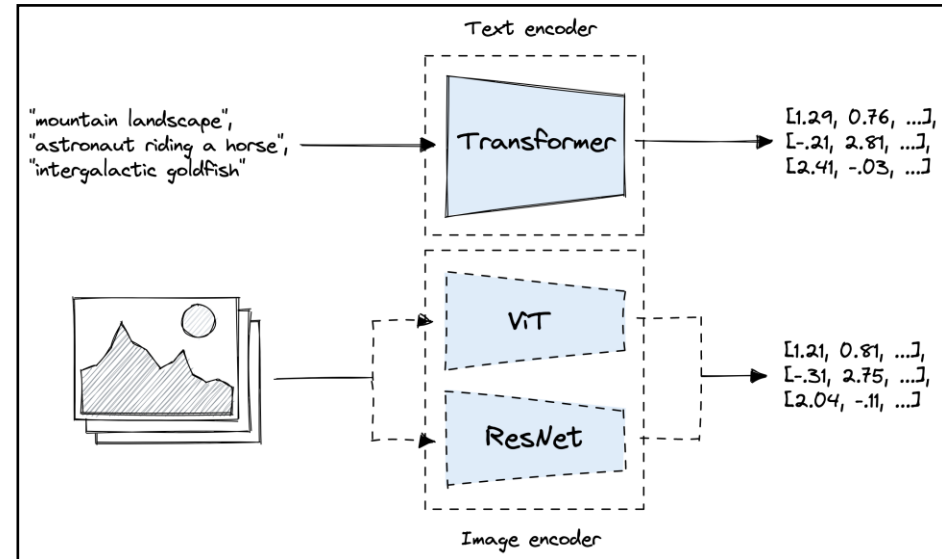


- Once you extract the 4096-D codes for all images, train a linear classifier (e.g. Linear SVM or Softmax classifier) for the **new dataset (your image dataset)**.

SOA of automatic feature extraction

Today the same workflow is often done with (next lesson)

- ResNet
- EfficientNet,
- ConvNeXt,
- Vision Transformers,
- CLIP-like encoders,
- or domain-specific pretrained models.

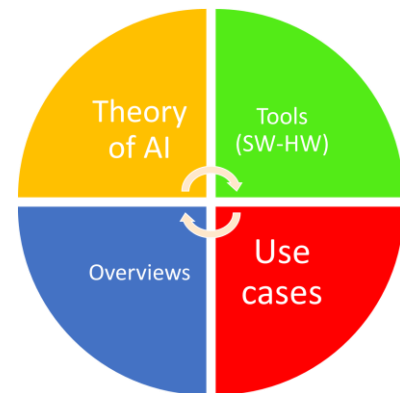




THEORY

Training and designing deep learning models

Facing the complexity of $>100M$ parameters



First of all...

DeepLearning

Design

(from previous lesson)

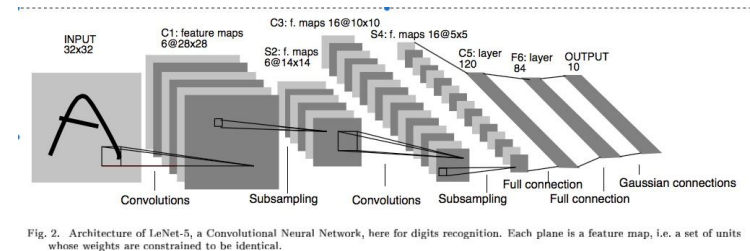
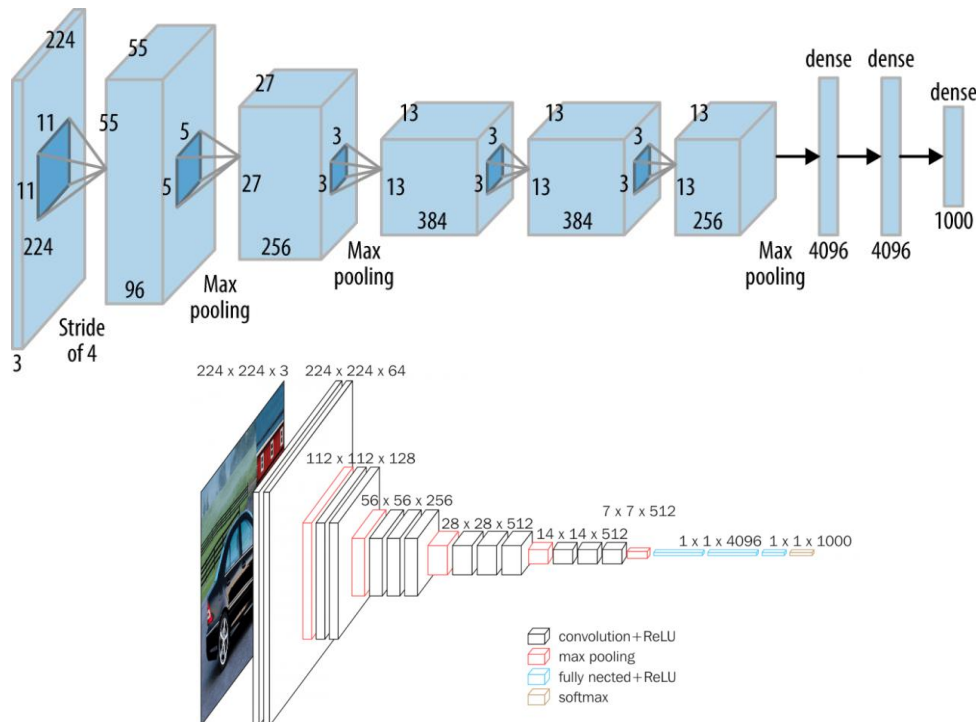


- While all **weights** are trained, the **structure (topology)** of the CNN is currently usually hand crafted with trial and error.
 - Number of total layers,
 - kernels types,
 - kernel sizes,
 - Size of sub-sampling (pooling) fields,
- Typically going along with layers from inputs → outputs
 - decrease size of kernels
 - increase number of feature maps

Deep Learning Training hints

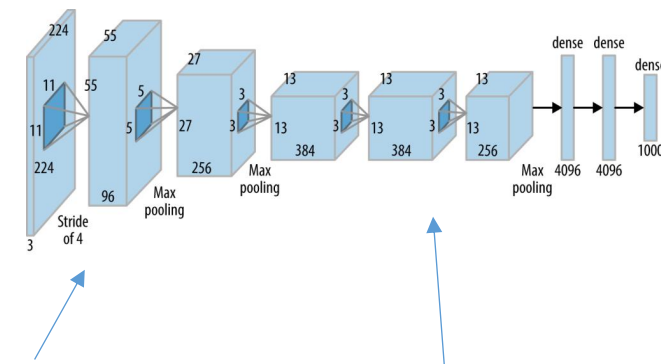
Trained with **Back Propagation (BP)**

- Use “weight gg” in each feature map (a sharing method for the weight matrix)
- Randomized initial weights through entire network
- Use pre-trained models



“Weight sharing” : The same convolutional kernel is reused across different spatial positions of the image. This strongly reduces the number of trainable parameters and makes the feature map translation-equivariant (hence helps to reduce the problem of overfitting).

Problems while training Deep Networks



- **Early layers** of MLP do not get trained well
 - Diffusion of Gradient – error attenuates as it propagates to earlier layers
 - Leads to very **slow training**
 - Exacerbated since top couple layers can usually learn any task "pretty well" and thus the error to earlier layers drops quickly as the top layers "mostly" solve the task
- **Lower layers** never get the opportunity to use their capacity to improve results, they just do a **random feature map**

Problems in training deep networks (2)

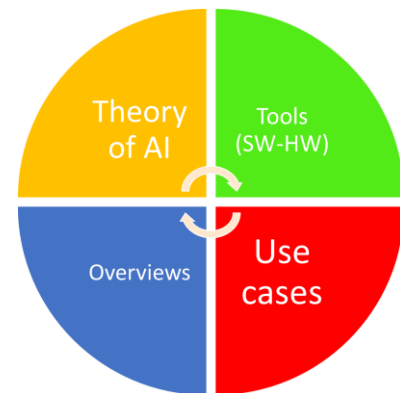
- Need a way for early layers to do effective work
- Instability of gradient in deep networks:
Vanishing or exploding gradient
 - Product of many terms, which unless “balanced” just right, is unstable
 - Either early or late layers stuck while “opposite” layers are learning
- Often **not enough labeled data available** while there may be lots of unlabeled data
 - Can we use unsupervised/semi-supervised approaches to take advantage of the unlabeled data
- **Deep networks tend to have more sensitive training issues and local minima problems than shallow networks during supervised training!**



THEORY

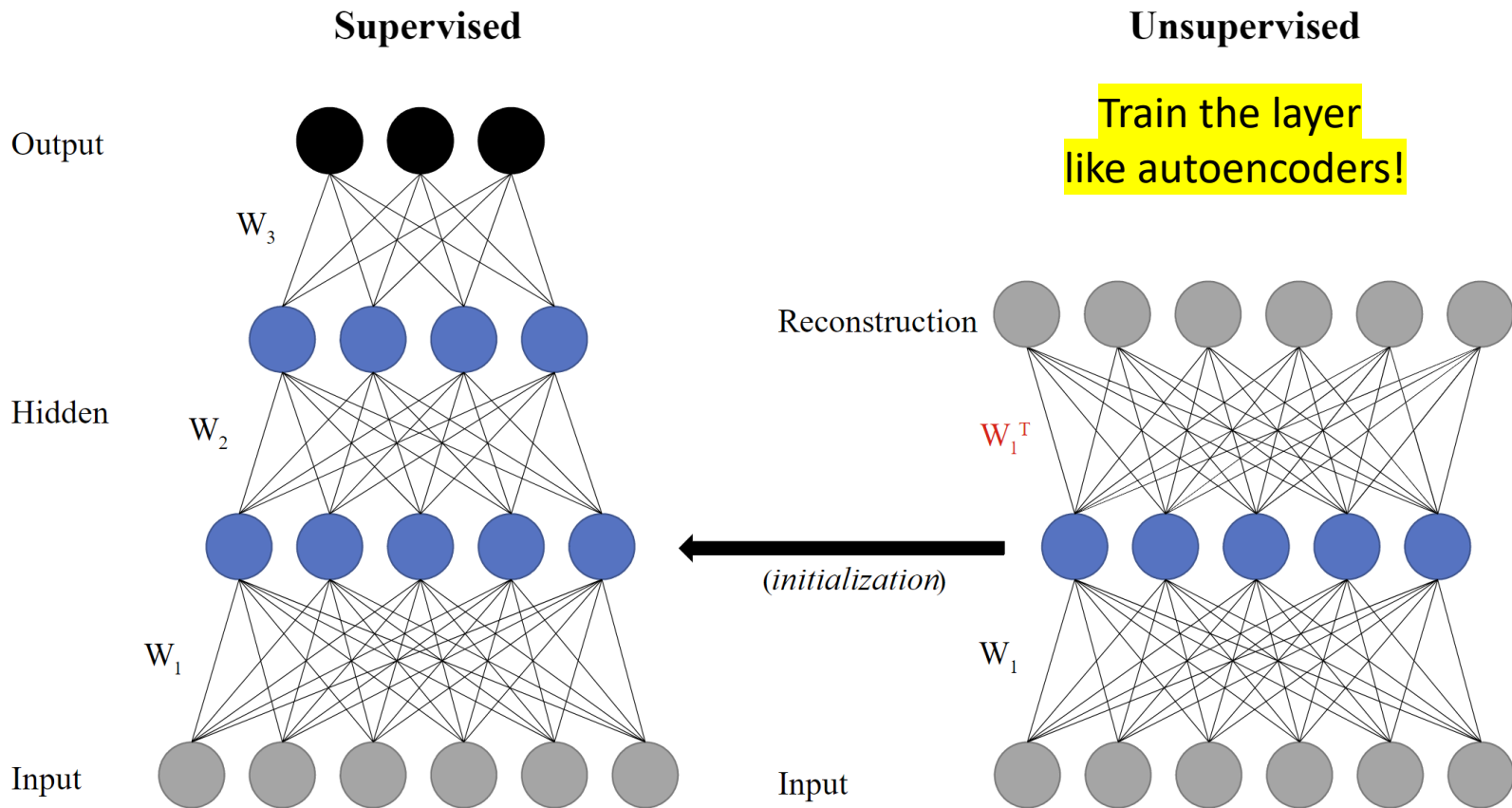
Greedy Layer-Wise Training

A very efficient solution to the problems in training of deep networks



Solution:

Use the unsupervised learning as initialization
→ export the layer

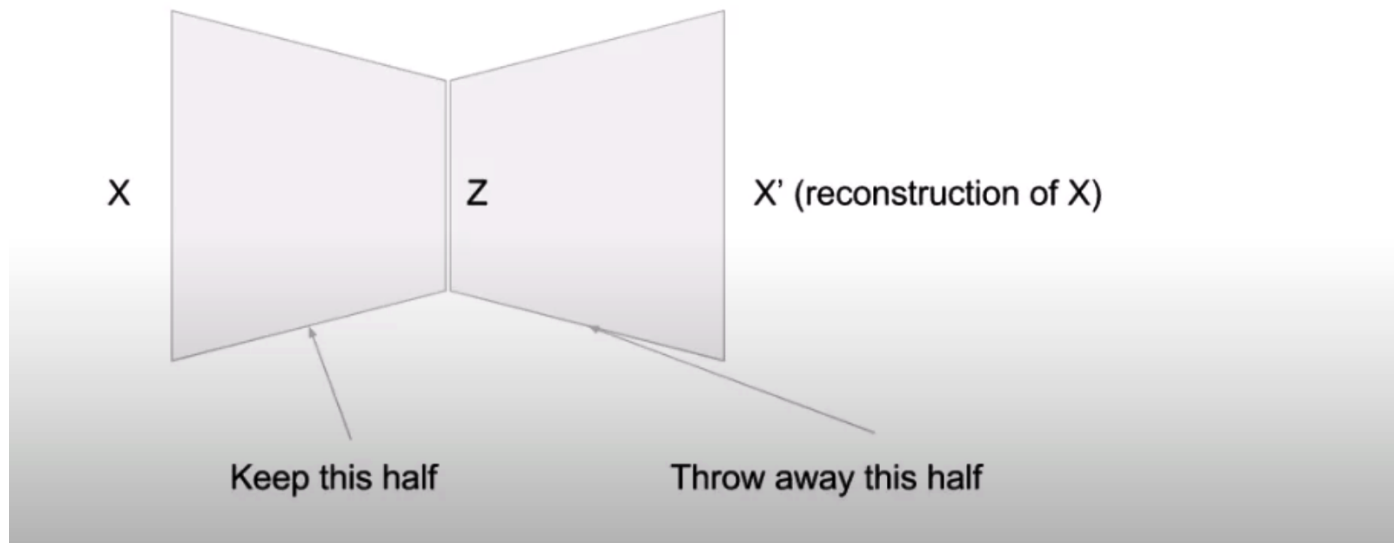


Solution:

Greedy Layer-Wise Training

- 1) Train first layer using your data without the labels (unsupervised)

Step 1) Train an autoencoder to reproduce the original images

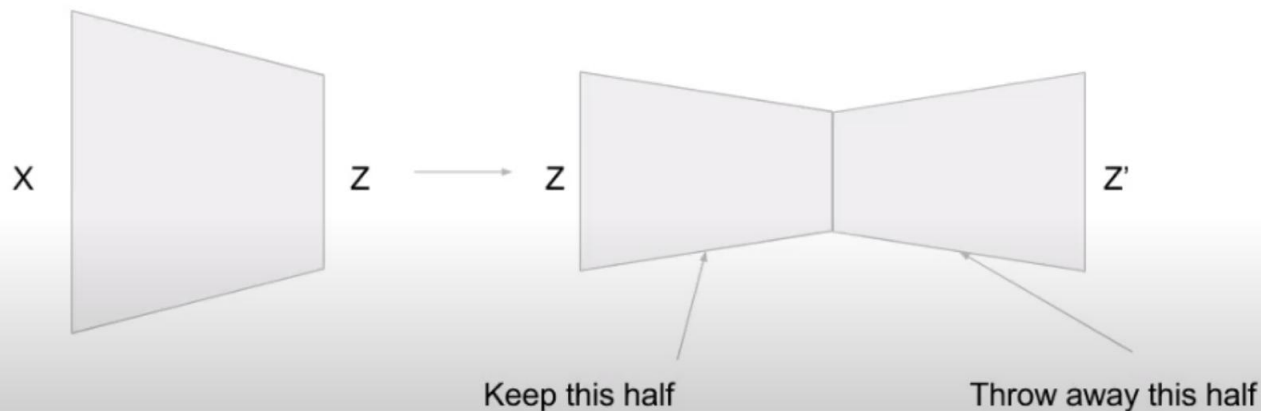


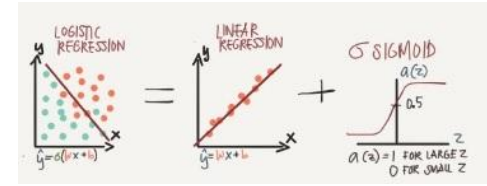
Solution:

Greedy Layer-Wise Training

- 2) Then freeze the first layer parameters and start training the second layer using the output of the first layer as the unsupervised input to the second layer

Step 2) Transform the images (X) using the first autoencoder (to Z), then train a 2nd autoencoder to reproduce Z (Z')



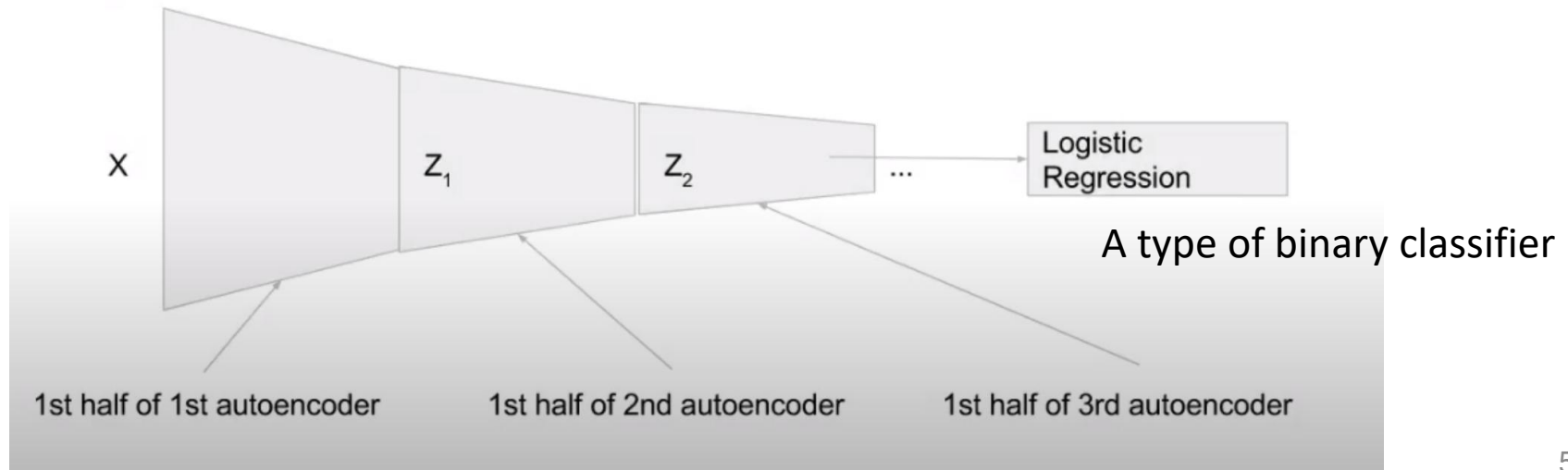


Solution:

Greedy Layer-Wise Training

- 3) Repeat this for as many layers as desired
- Use the outputs of the final layer as inputs to a supervised layer/model and train the last supervised layer(s) (leave early weights frozen)

Step 3) Attach 1st half of 2nd autoencoder to 1st half of 1st autoencoder, repeat as desired

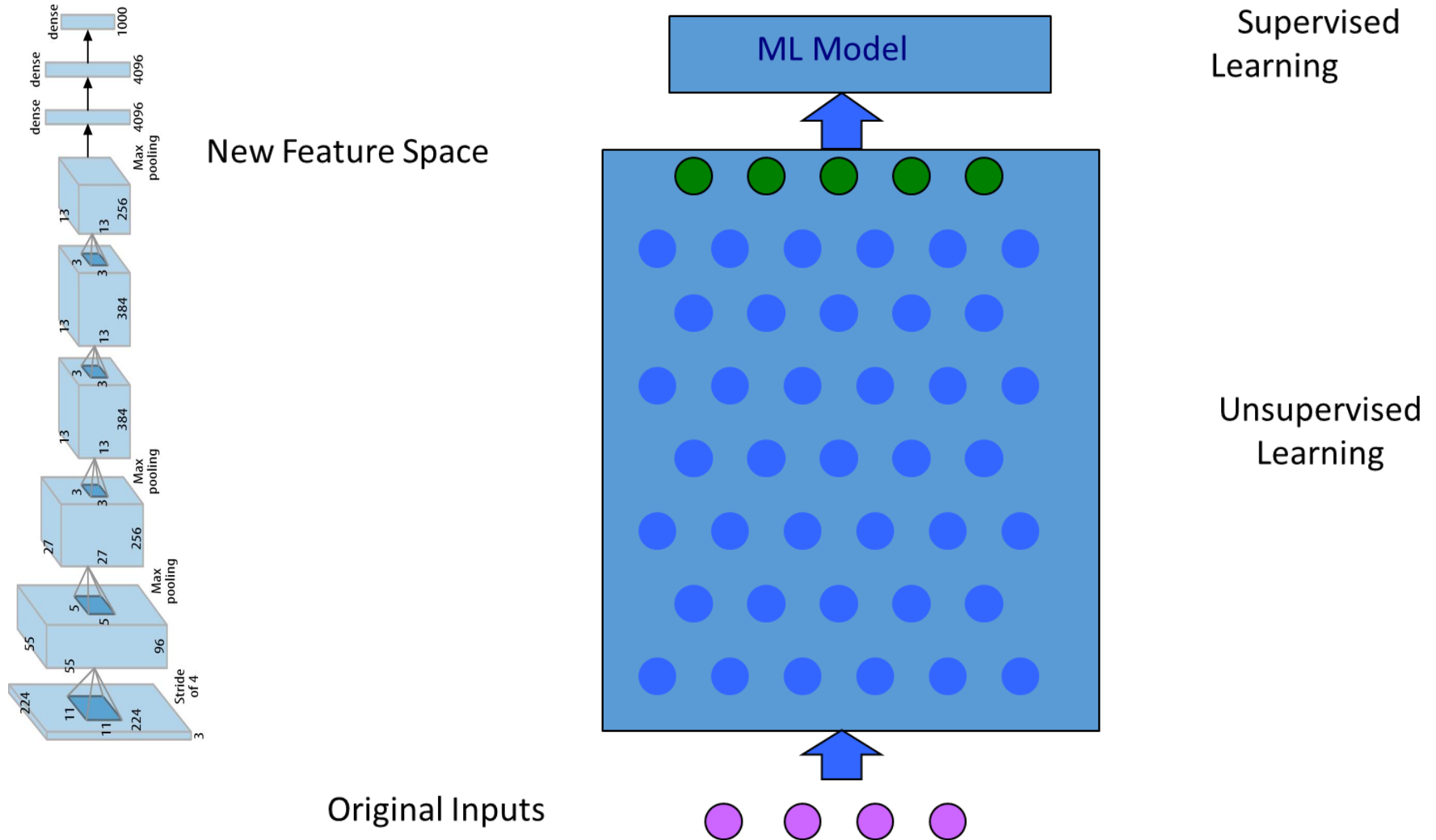


Greedy Layer-Wise Training:

resuming

1. **Train** first layer using your data without the labels (unsupervised)
2. Then **freeze** the first layer parameters and start training the second layer using the output of the first layer as the unsupervised input to the second layer
3. **Repeat** this for as many layers as desired
4. Use the outputs of the final layer as inputs to a supervised layer/model and **train the last supervised layer(s)** (leave early weights frozen)
5. Unfreeze all weights and **fine tune the full network** by training with a supervised approach, given the pre-training weight settings

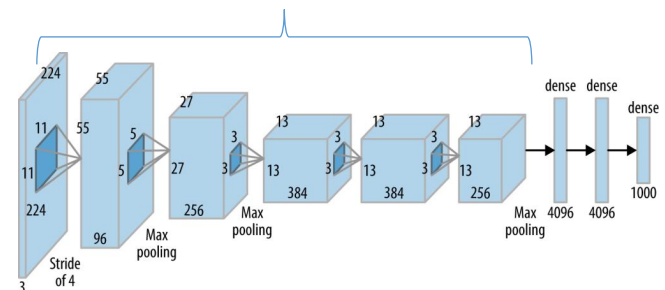
Deep Net with Greedy Layer Wise Training



Greedy Layer-Wise Training

- Greedy layer-wise training avoids many of the problems of trying to train a deep net in a supervised fashion
 - **Each layer gets full learning focus** in its turn since it is the only current "top" layer
 - Can take advantage of **unlabeled data**
 - When you finally **tune the entire network** with supervised training the network weights have already been adjusted so that you are in a good error basin and just need fine tuning. This helps with problems of
 - Ineffective early layer learning
 - Deep network local minima

We solved the training problems here





Generative Adversarial Networks (GAN)

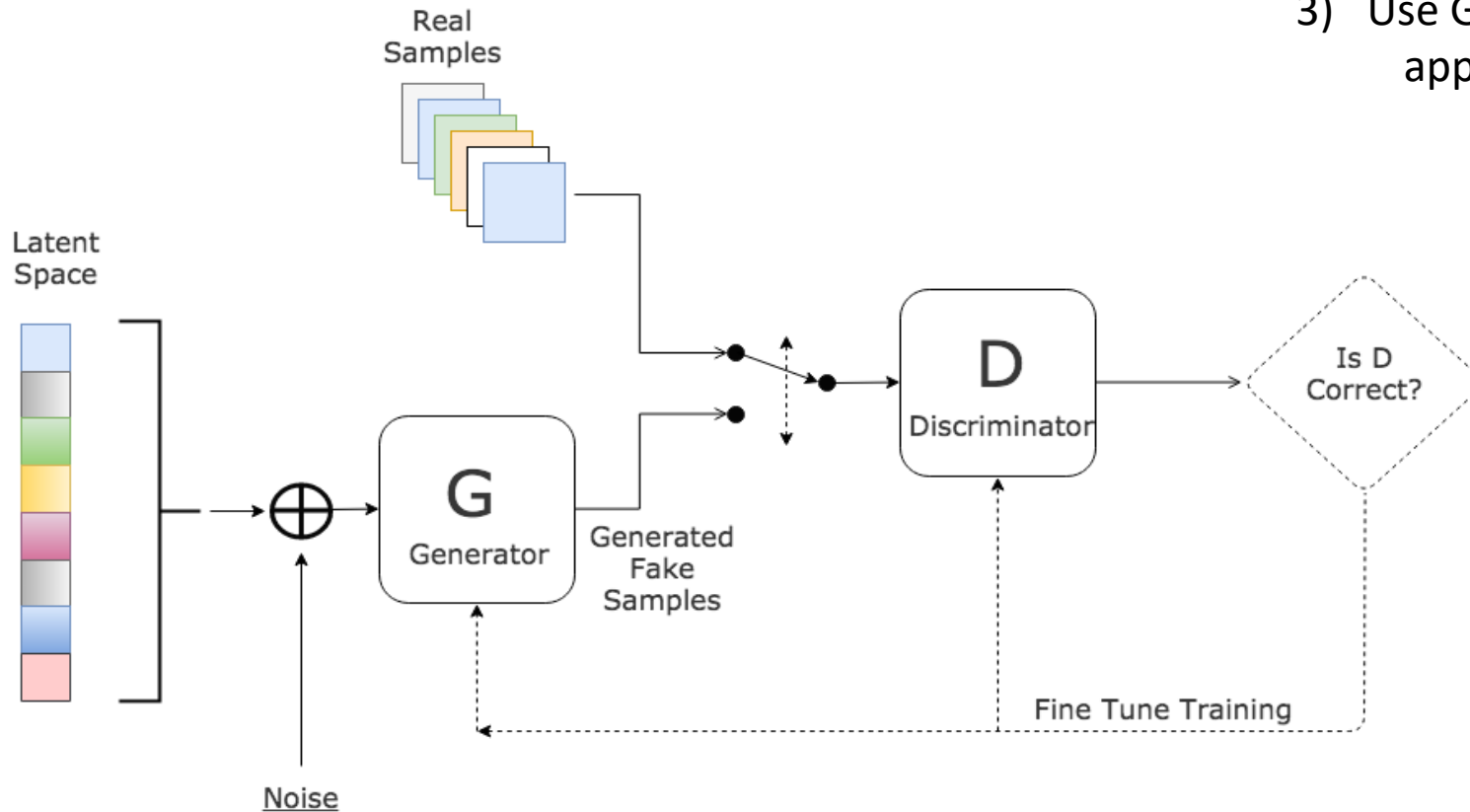
Creating a pattern/image from a random number.

A different point of view to train Deep Neural Networks

GANs Net

Generative Adversarial Network

- 1) A known dataset is used as initial training data for the discriminator.
- 2) GAN Training
- 3) Use G() in your application



Real or created by a GAN?



GAN

- Data augmentation
- Attacks
- Simulations of different conditions (style transfer)
- ...

Age progression

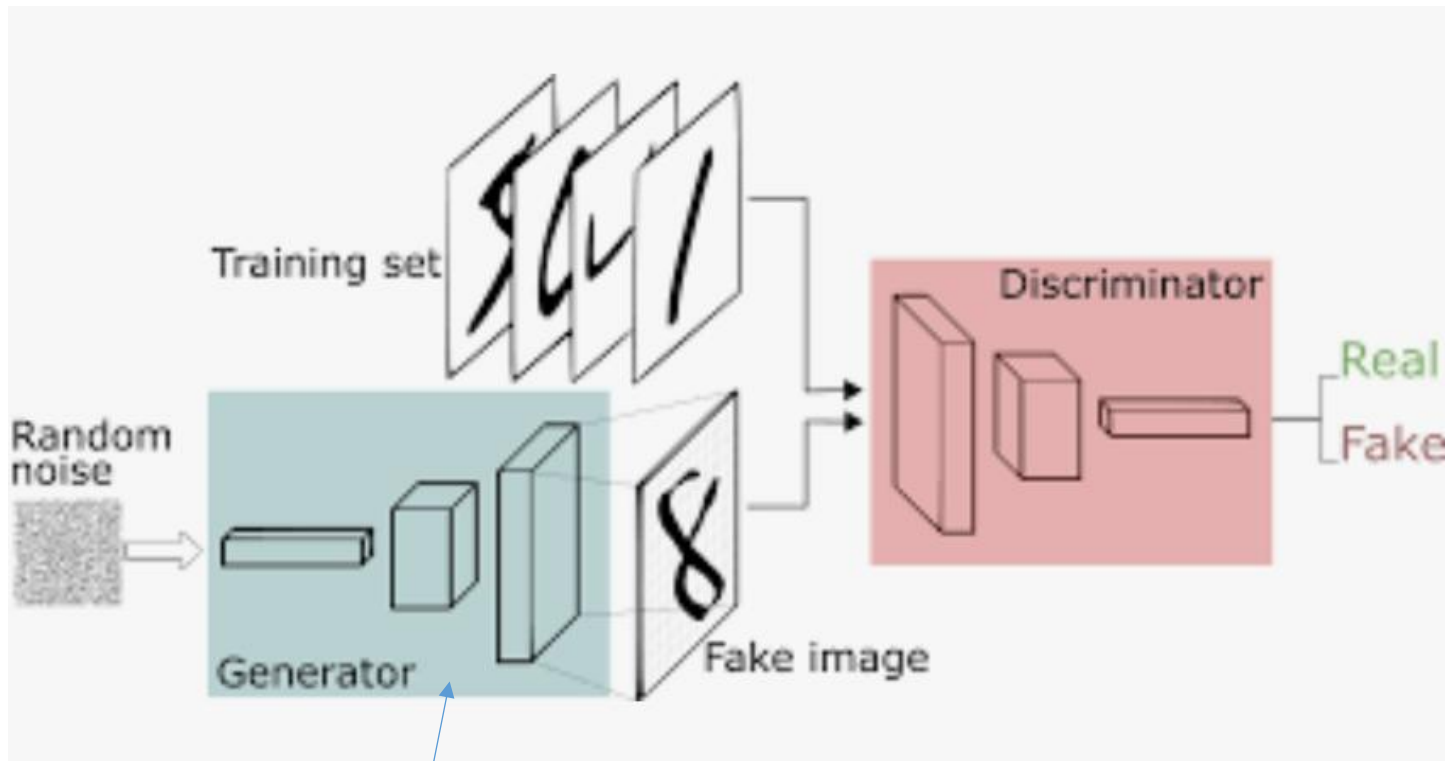


Day/Night

Winter/
Summer

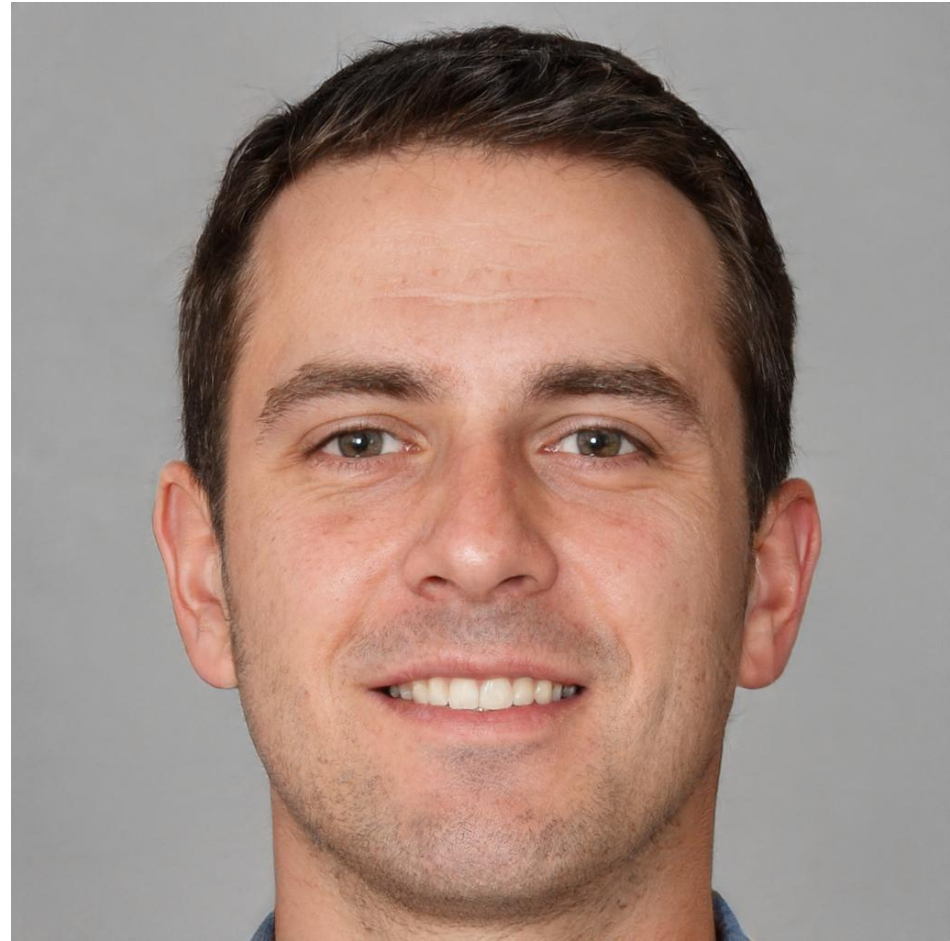
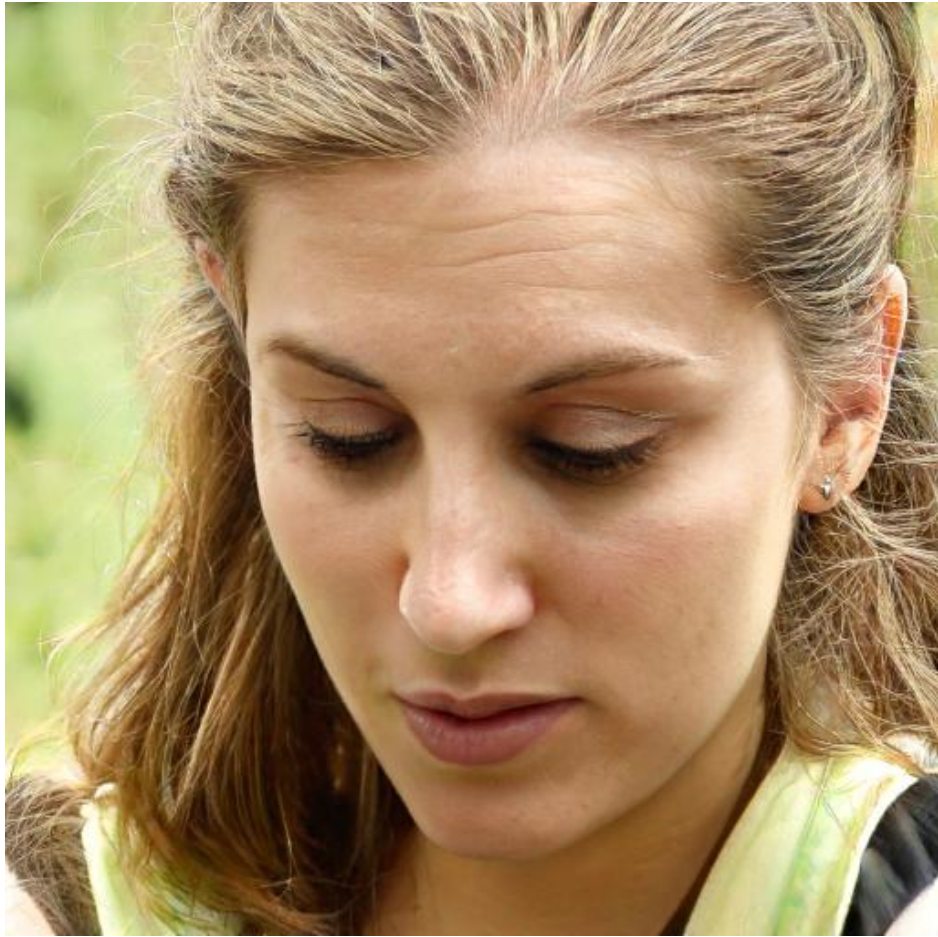
GANN:

Two networks competing



Note the «shape» of the Convolutional layers

Real or NN-generated?

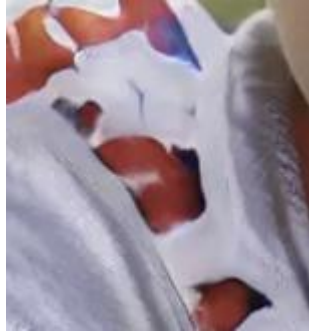


<https://www.thispersondoesnotexist.com/>

Example of fake faces



«Gestalt» is still missing...



The analysis of the context is revealing the GAN errors

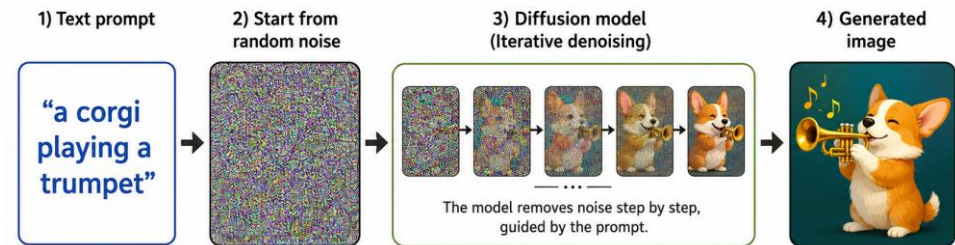
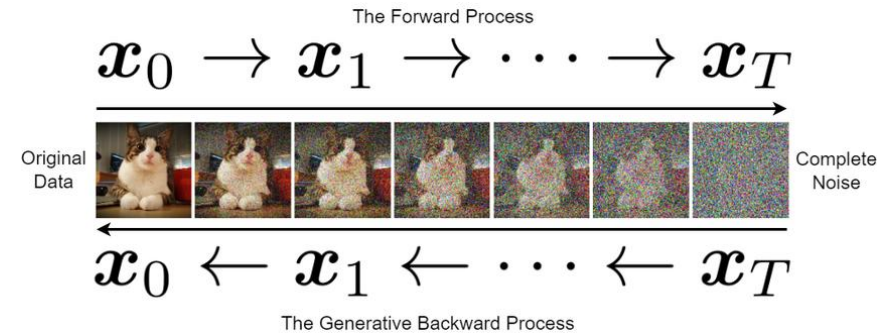
The fusion of the features of different levels is not (yet) perfectly realistic

Result using StyleGAN2 (Dec 2019) are improved...



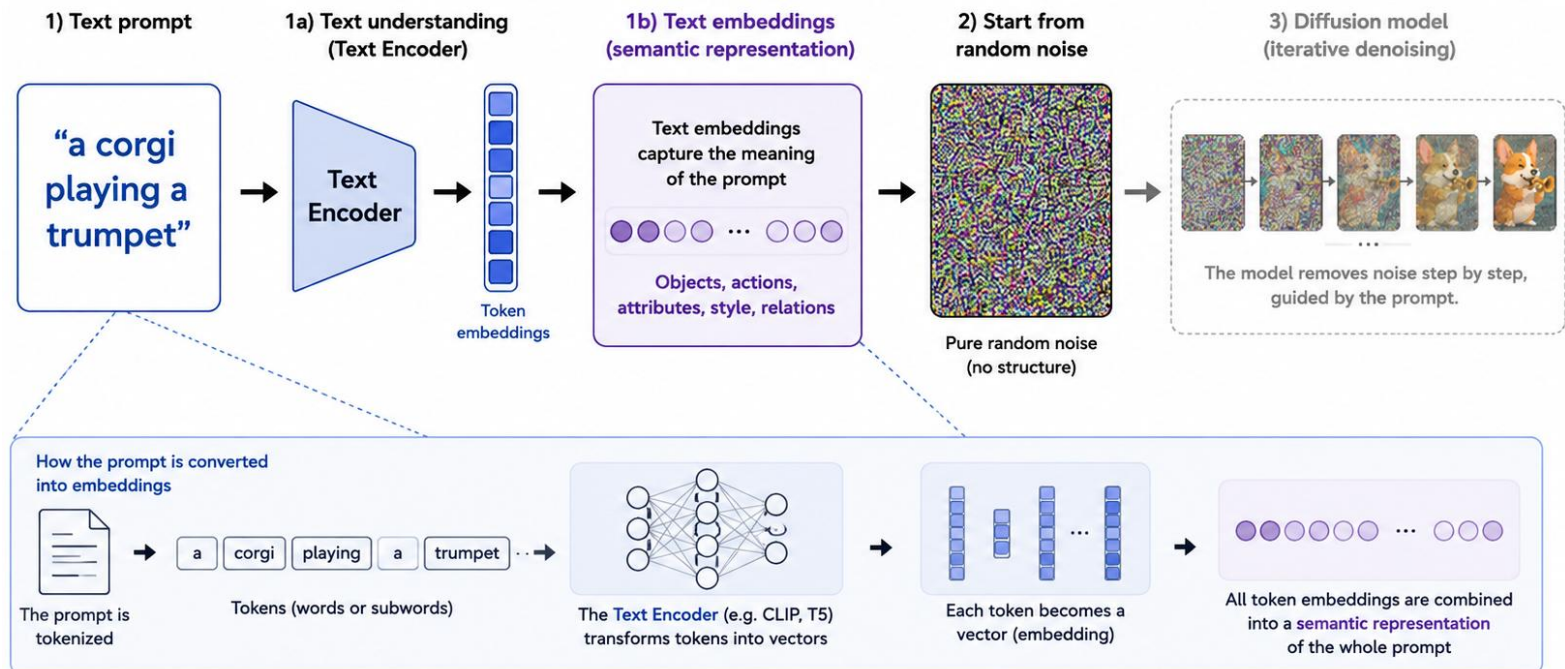
Diffusion models (denoise)

- Diffusion models generate data by learning how to reverse a gradual noising process.
- During **training** the model observes images progressively corrupted by noise and learns to predict how to remove that noise step by step.
- At **generation** time the process starts from random noise and iteratively denoises it until a realistic image is produced.



Diffusion models (denoise)

- The text encoder converts the prompt into embeddings. These embeddings condition the diffusion model, guiding each denoising step toward an image that matches the prompt. CLIP is commonly used in models such as Stable Diffusion v1, but other text encoders such as T5 are also used in modern systems.
- They are now one of the dominant approaches for high-quality image generation, image editing, inpainting, super-resolution, and text-to-image systems. **We are using them for data augmentation!**



Sony World Photography Awards 2023

BBC 18/04/2023

German artist Boris Eldagsen's entry, entitled Pseudomnesia: The Electrician, won the creative open category at last week's Sony World Photography Award.

Winner refuses award after revealing AI creation



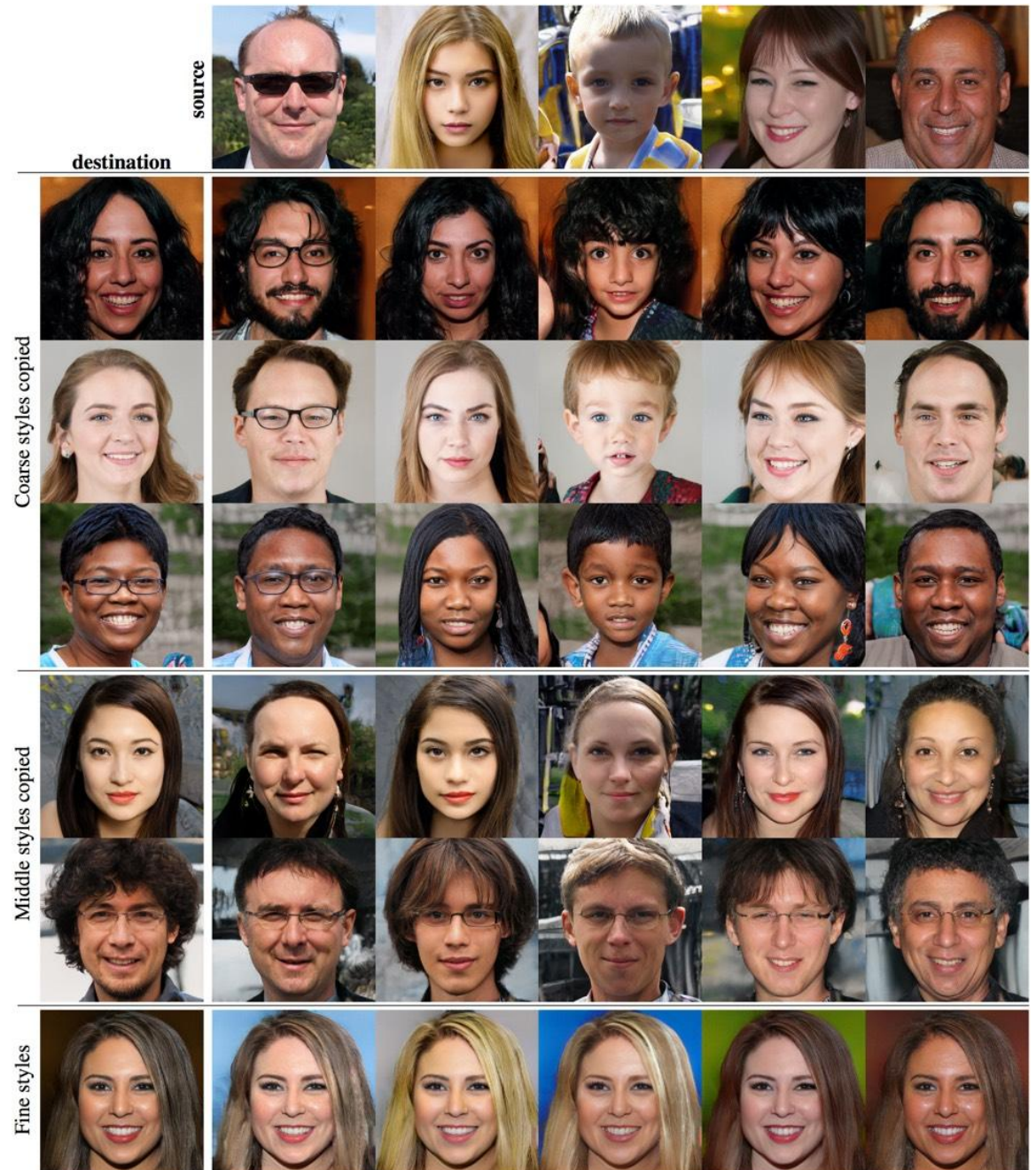


GESTALT IS MISSING (but, what about today?!)

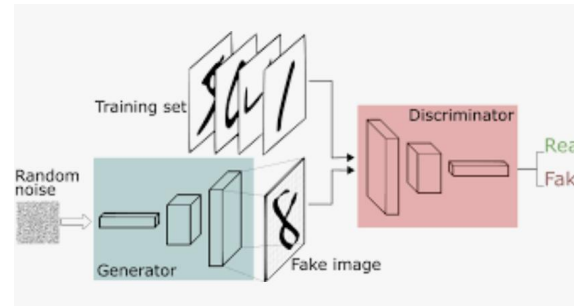
Style transfer by CNN (GAN)

Take your time to appreciate the combinations...

In the following lessons we will study how to do it.



GAN art



October 25th 2018

Christie's sold a portrait for \$432,000 that had been generated by a GANN, based on open-source code written by Robbie Barrat of Stanford.

(congratulations!)



GAN for

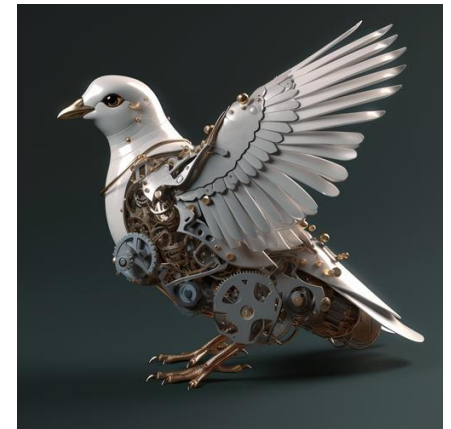
Not only faces... you can use GAN to create

- Voices → Text-to-Speech
- Biometrics (e.g., fingerprints)
- New Datasets
 - Generating Realistic Training Data: GANs can generate synthetic data that can be used for training machine learning models, especially when privacy issues prevent the use of real data.

GAN for

And solve different applications

- Image Synthesis
- Data Augmentation
- Super Resolution
- Text to Image Synthesis
- Image-to-Image Translation
- De-Oldifying Images and Videos
- Voice Generation and Conversion



“mechanical dove”
created with
Midjourney



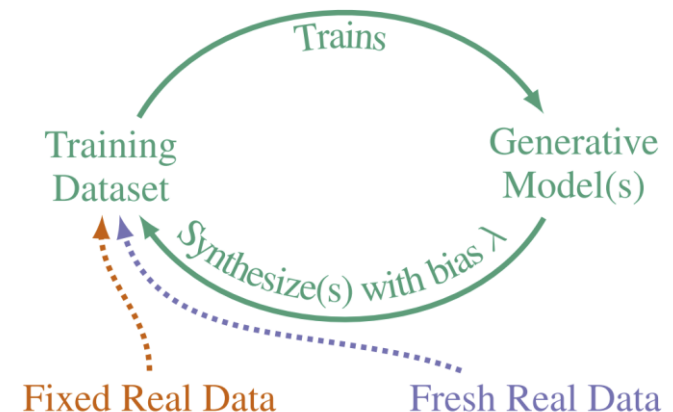
«Cows eating other cows»
got mad..

The MAD Effect

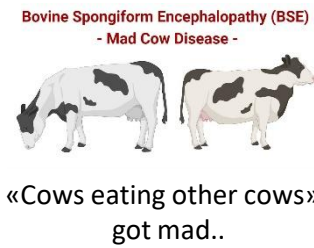
Model autophagy disorder

The worst enemy of artificial intelligence could be artificial intelligence itself

- A study from Rice and Stanford University revealed that training AI models through synthetic data seems to cause their erosion, at least in the long term.
- The term indicates this apparent 'self-allergy' of the software is Mad, an acronym for Model autophagy disorder."
- Many AIs are thus unknowingly trained on increasing amounts of already synthesized data"



Reduction of the Model autophagy disorder



Synthetic data requires provenance, filtering, diversity control, and validation

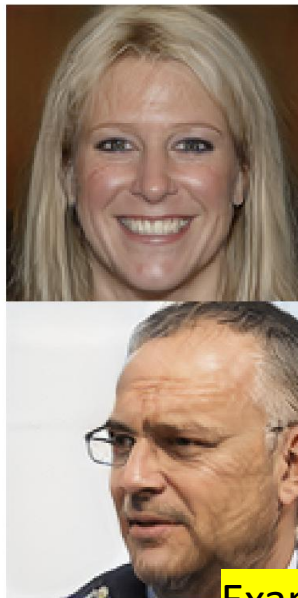
Conclusion:

without enough fresh real data in each generation of an autophagous loop, future generative models are doomed to have their quality (precision) or diversity (recall) progressively decrease

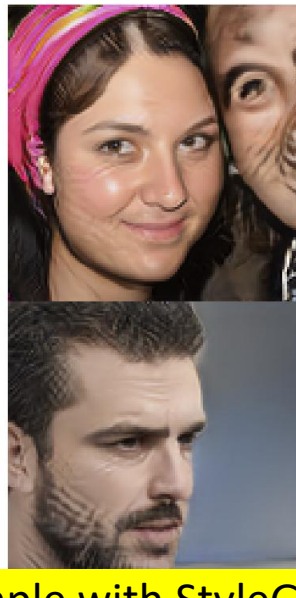
Generation $t = 1$



$t = 3$



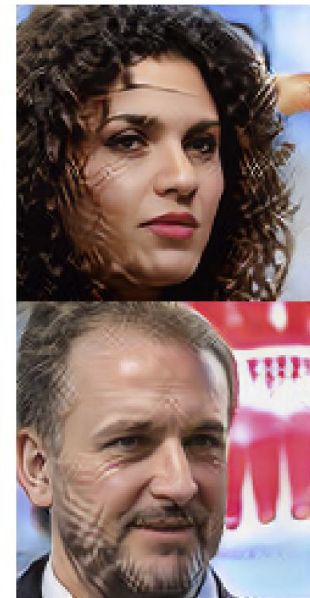
$t = 5$



$t = 7$



$t = 9$



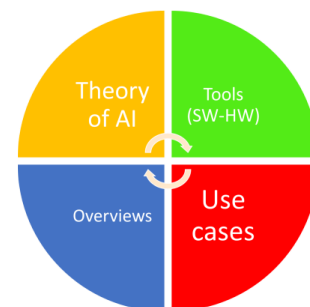
Example with StyleGAN-2 re-trains



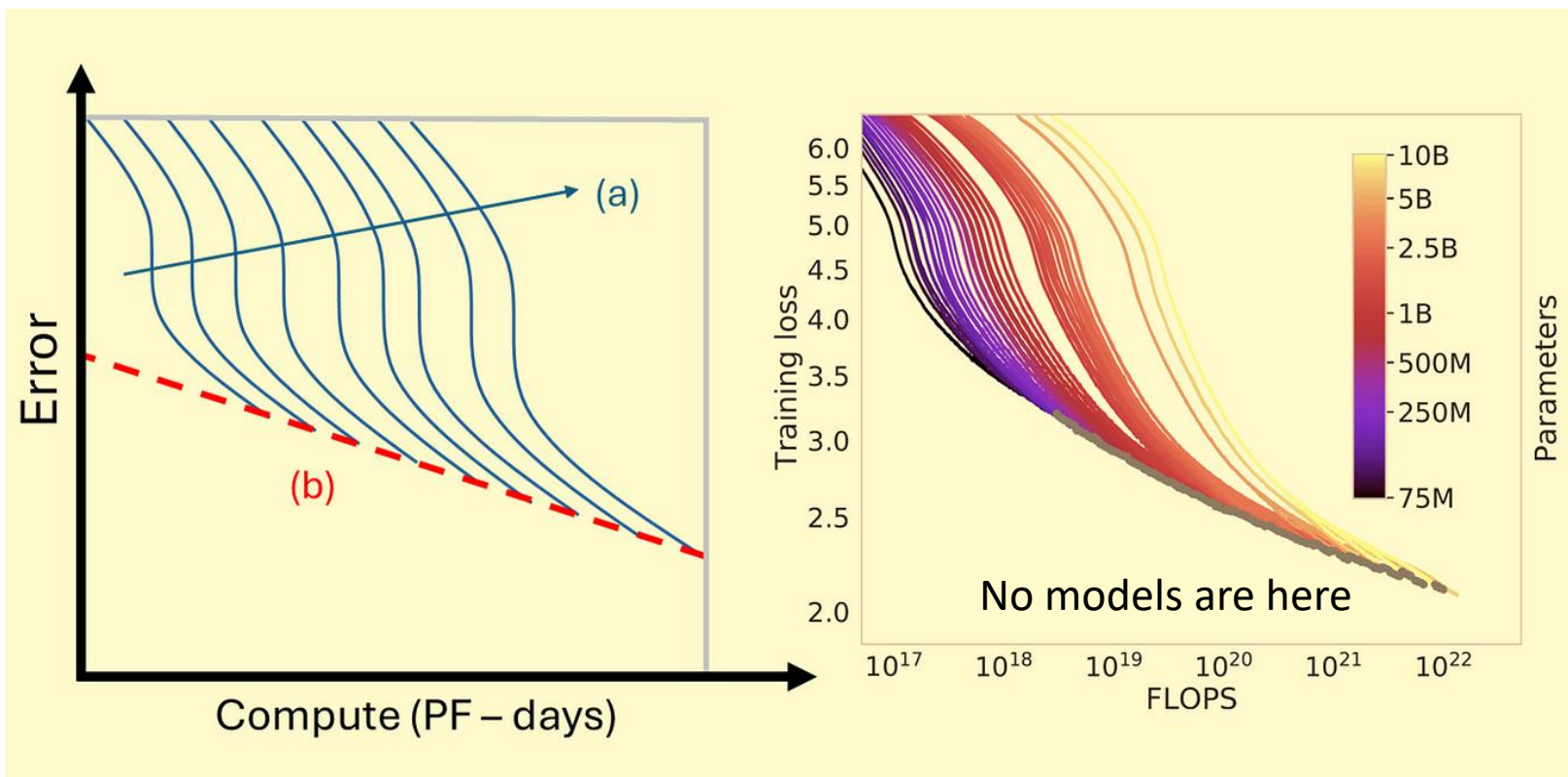
THEORY

Current boundaries in ML

More data? More Parameter?
Is it the correct way to get better systems?



Efficient Comput Frontier



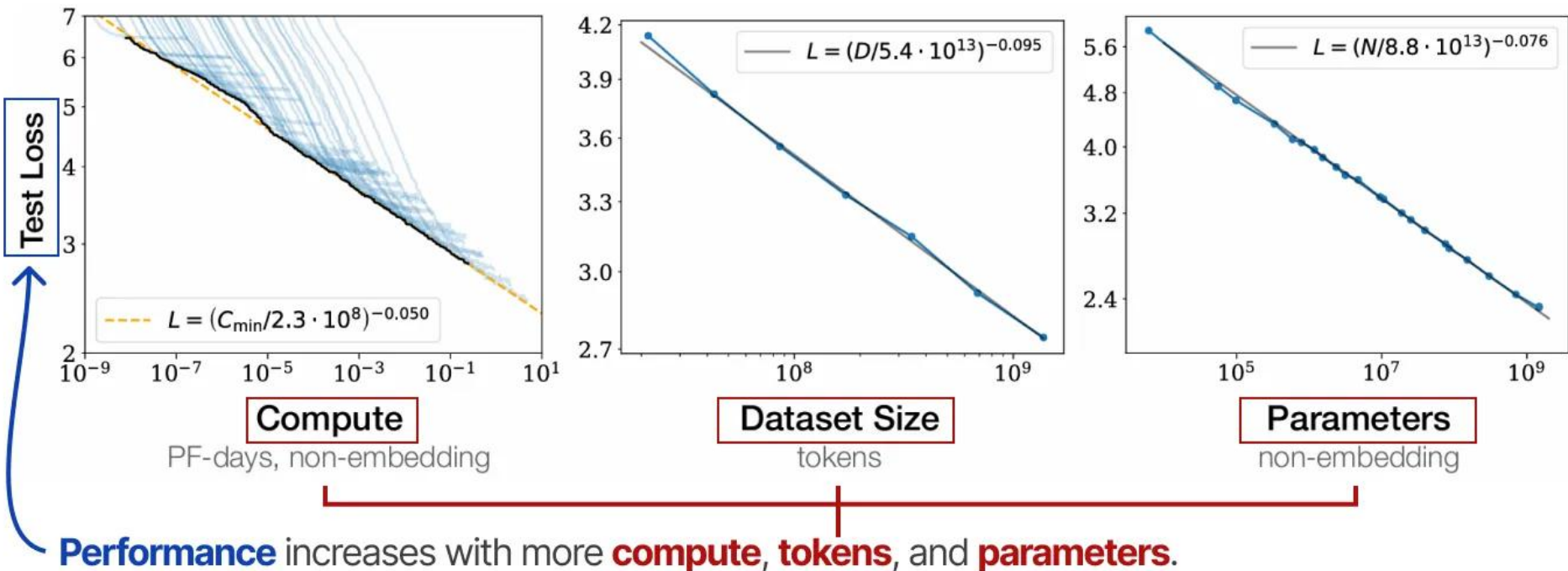
Exponential increase in training compute for OpenAI's GPT models from 2018 to 2023

The blue curves (a) illustrate increasing model sizes that require more compute to achieve lower error rates.

The red dashed line (b) represents the efficient compute frontier, beyond which additional compute yields diminishing returns in performance improvements.

Scaling laws for neural language models (LLMs)

Performance increases (error goes down) with different measures of computation (longer training, dataset size, and parameter size). See the trend!

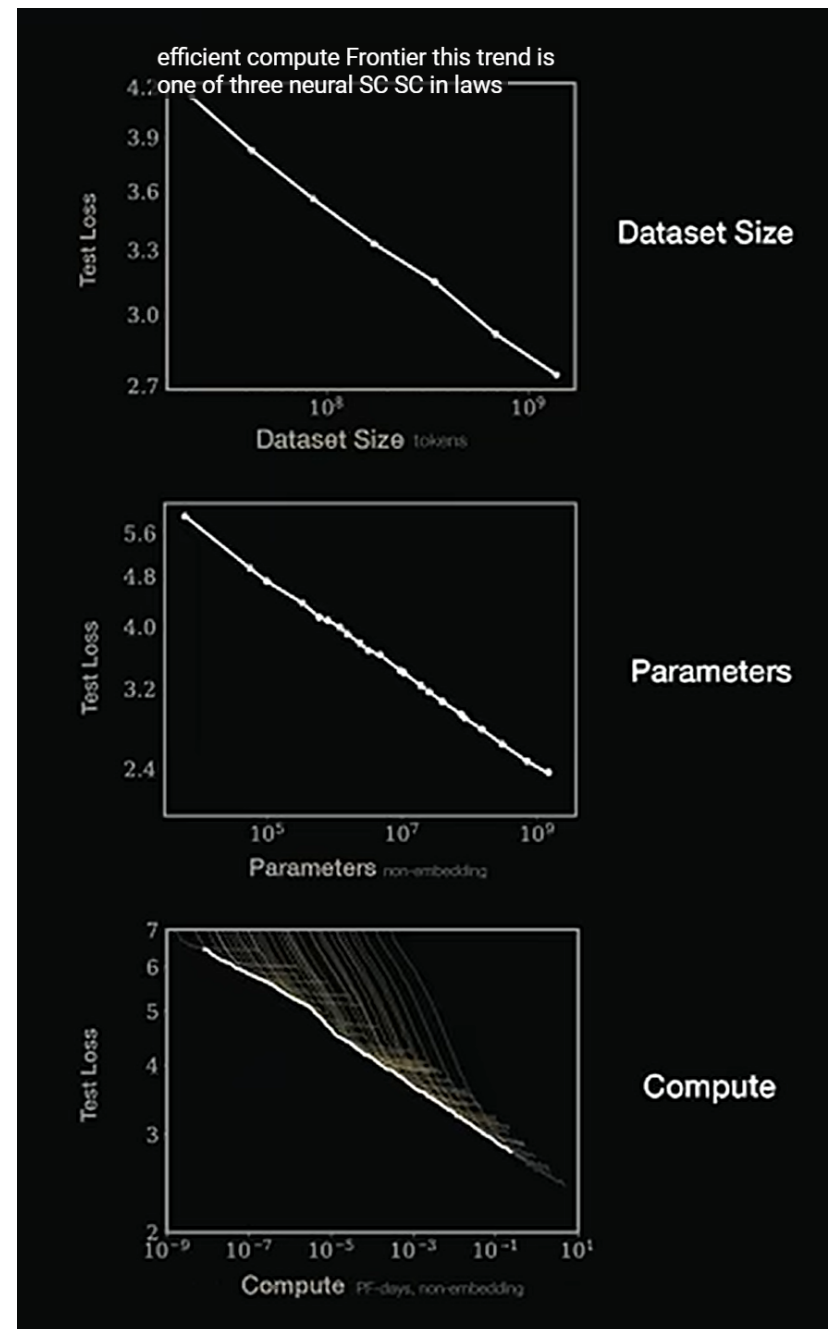


Limits for ViTs

Visual Transformers (also known as **Vision Transformers**, or **ViTs**) are common deep neural network used to process images.

They use a **transformer architecture**, originally developed for natural language processing (e.g., in models like BERT or GPT), to process **visual data**.

The error/loss in training in ViTs has a trend quite similar to one found in LLMs



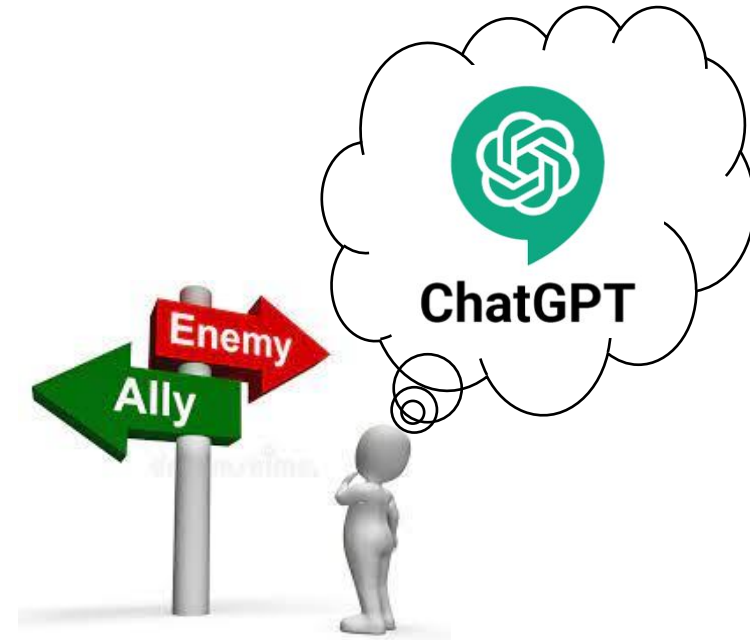


LLM and Machine Learning Design

Discussion of what can do a tool such as ChatGPT to help the work of a Machine Learning Designer

Enemy or Ally?

- As all AI language model, ChatGPT can be seen as an ally for data analysts and ML designers rather than an enemy
- It's crucial to exercise critical thinking, validate results, and independently verify the information provided
- As a language model, it does not have access to real-time or domain-specific data, and its responses are based on patterns learned from pre-existing information



ChatGPT can leverage your work

You can leverage ChatGPT capabilities in your work by different tasks:

1. Exploratory Data Analysis, Data Visualization
2. Data Preprocessing, Feature Selection and Dimensionality Reduction
3. Algorithm Selection
4. Help in specific applications such as “Time Series Analysis” and “Natural Language Processing” (NLP)
5. Model Development, Deep Learning, Model Evaluation and Interpretation
6. Code Assistance, Troubleshooting and Debugging
7. Research and Conceptual Understanding, Research and Literature Review, Inspiration and Innovation
8. etc....

ChatGPT can leverage your work

You can leverage ChatGPT capabilities in your work by different tasks:

1. **Exploratory Data Analysis**
analyze and provide insights about your datasets. Understand the data distribution, identify patterns, detect outliers, and explore relationships between variables
2. **Data Preprocessing**
Preprocessing tasks such as data cleaning, handling missing values, feature scaling, and encoding categorical variables.
Guidance or specific code examples to handle these preprocessing steps effectively
3. **Algorithm Selection**
Selecting the appropriate machine learning algorithms for your problem, providing insights and recommendations based on your dataset characteristics and objectives.
Explains the pros and cons of different algorithms and help you make informed decisions

ChatGPT can leverage your work

4. Model Development

Whether you are working on classification, regression, clustering, or other machine learning tasks, you can ask for help in model development

Guidance on implementing machine learning models, optimizing hyperparameters, and evaluating model performance

5. Code Assistance

If you encounter coding challenges or need assistance with specific programming tasks it provides code snippets or explain programming concepts in languages like Python, R, or MATLAB

6. Research and Conceptual Understanding

If you require explanations or insights into statistical concepts, machine learning techniques, or data science best practices, it can provide explanations and help enhance your understanding

Create a data dictionary

- >> give me a data dictionary of the iris dataset



give me a data dictionary of the iris dataset



Here's an example of a data dictionary for the Iris dataset, a popular dataset used in machine learning and statistics:

Dataset: Iris

Description: The Iris dataset contains measurements of various features of Iris flowers. It is commonly used for classification and clustering tasks.

Fields:

1. Sepal Length (cm)
 - Data Type: Numeric (continuous)
 - Description: The length of the sepal (the outermost part of the flower) in centimeters.
2. Sepal Width (cm)
 - Data Type: Numeric (continuous)
 - Description: The width of the sepal in centimeters.
3. Petal Length (cm)
 - Data Type: Numeric (continuous)

Regenerate response

....

The last field, "Species," is a **categorical variable** that represents the species of the Iris flower with three distinct categories: Setosa, Versicolor, and Virginica.

Dictionary of my data..

- >> give me a data dictionary of this dataset:

```
Can you give me a data dictionary for this dataset: State Total.Rate Total.Number Total.Population
Rates.Age.< 18 Rates.Age.18-45 Rates.Age.45-64 Alabama 214.2 71529 33387205 2 18.5 244.7 Alaska
128.1 6361 4966180 1.7 11.8 170.9 Arizona 165.6 74286 44845598 2.5 13.6 173.6 Arkansas 223.9 45627
20382448 2.3 17.6 250.1 California 150.9 393980 261135696 2.6 13.7 163.7 Colorado 139 49035
35267734 1.9 11.7 149.6
```

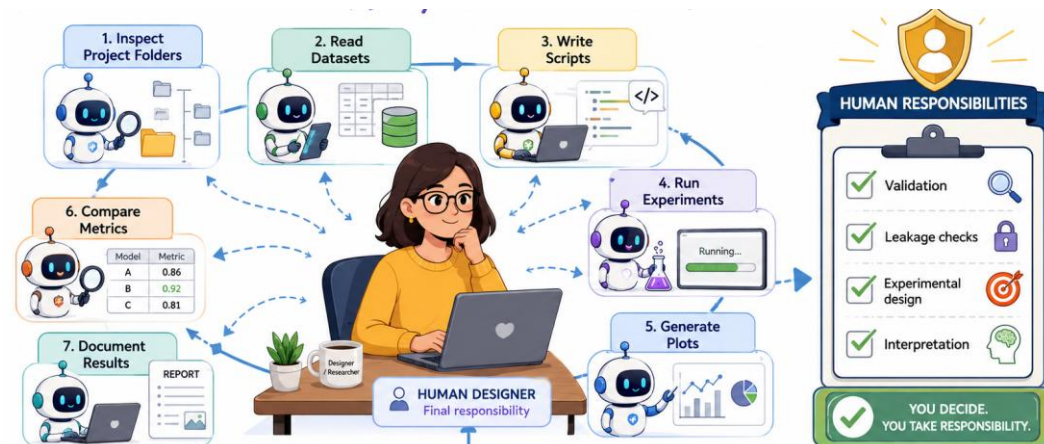


Column Name	Data Type	Description
State	string	State name
Total.Rate	decimal	Total rate
Total.Number	integer	Total number
Total.Population	integer	Total population
Rates.Age.< 18	decimal	Rate for age less than 18
Rates.Age.18-45	decimal	Rate for age between 18 and 45
Rates.Age.45-64	decimal	Rate for age between 45 and 64

Is using the context to create it!

Design ML with agents

- LMs are not only chat assistants. Agentic systems can inspect project folders, read datasets, write scripts, run experiments, generate plots, compare metrics, and document results.
- The designer remains responsible for validation, leakage checks, experimental design, and interpretation.



Agents and the “gold rush”

I final thought about...

- During the goldrush (a period of mass migration in the pursuit of gold in various parts of the world, particularly in the 19th century) most participants did not find wealth, but instead faced challenging circumstances or even incurred losses
- Some individuals earned substantial profits not through direct gold extraction, but through ancillary activities
- This “historical recall” can help us to reframe what such *Large-Scale Language Models* and Agents can do/help in your work. For ex. considering new activities helped/based/improved and not “against of” or that they can “substituted by” ...



Main points



- Autoencoders
- Training and design of deep learning models
 - Fine Tuning
 - Transfer Knowledge
 - Data augmentation
- Catastrophic forgetting
- Automatic Feature Extraction
- Problems of the training of deep learning models.
- A solution: Greedy Layer-Wise Training
- Generative Adversarial Networks
 - Generation of realistic synthetic images
 - Model autophagy disorder for generative models
- Scaling laws for neural language models
- Discussion on LLM (e.g., ChatGPT) and the Machine Learning Design